

Universidad
Rey Juan Carlos

Escuela Técnica Superior
de Ingeniería Informática

Grado en Diseño y Desarrollo de Videojuegos

Curso 2025-2026

Trabajo Fin de Grado

**REALIZACIÓN DE UN JUEGO MULTIGÉNERO:
RESET GAME**

Autor: Mario Martín Escribano

Tutor: Nikola Hristov Kalamov



©2025 Mario Martín Escribano
Algunos derechos reservados
Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional” de Creative Commons,
disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Agradecimientos

Breves agradecimientos o dedicatoria.

Resumen

En el panorama actual de la industria de los videojuegos, el mercado se caracteriza por una marcada especialización donde los títulos suelen limitarse a un único bucle de juego básico. Frente a esta fragmentación, surge la oportunidad de diseñar arquitecturas de software capaces de integrar y dividir géneros lúdicos tradicionalmente discordantes bajo un mismo sistema modular. En este contexto nace *Reset*, un proyecto desarrollado con el propósito de fusionar dinámicamente la perspectiva cenital (*Top-Down*) orientada a la acción y la supervivencia, y la perspectiva lateral (*Side-Scroller*) enfocada en las plataformas y la precisión técnica, conectando ambos mundos mediante un nexo narrativo y espacial central.

El desarrollo de este trabajo se ha abordado desde una metodología de trabajo multidisciplinar para dar vida a una demo jugable (*Vertical Slice*), cubriendo de forma integrada todas las fases de la creación del producto. El proceso comenzó con la conceptualización y redacción del Documento de Diseño de Juego (GDD) y la estructuración del guion narrativo. Posteriormente, en la fase de ingeniería de software, se programaron desde cero utilizando el lenguaje C# bajo el motor de juego Unity los controladores de físicas y de movimiento del personaje, los sistemas de inteligencia artificial para el comportamiento autónomo de los enemigos y una arquitectura modular para la persistencia de datos y guardado. Todo este desarrollo lógico se complementó con un dedicado esfuerzo en la dirección artística, puliendo los gráficos en estilo *pixel art*, las animaciones y el diseño de la interfaz y la experiencia de usuario (UI/UX), consolidando un prototipo jugable sólido que demuestra la viabilidad de unificar el desarrollo informático con el diseño de juego (*Game Design*).

Palabras clave:

- Videojuegos
- Hibridación de géneros
- Game Design
- Experiencia de usuario (UI/UX)
- Pixel Art
- Vertical Slice
- Inteligencia Artificial

- Persistencia de datos
- C#
- Unity

Índice de contenidos

Índice de tablas

Índice de figuras

Índice de códigos

1. Introducción	1
1.0.1. Contexto del Proyecto	1
1.0.2. El panorama actual y la barrera de entrada a los géneros	2
1.0.3. El reto del diseño: Cohesión frente a fragmentación	3
1.0.4. El reto de jugabilidad: Imposibilidad frente a aburrimiento	3
1.1. Estado del arte	4
1.1.1. El origen y la diversificación de los géneros en el videojuego	4
1.1.2. La historia reciente y la diversificación de subgéneros	6
1.1.3. Justificación de la arquitectura de diseño en RESET: Modelos de hibridación por paradigmas	8
2. Objetivos	12
3. Tecnologías, Herramientas y Metodologías	15
3.1. Motor de Videojuegos: Unity 6	15
3.1.1. Justificación Técnica y Elección del Motor	16
3.1.2. Rol en el Proyecto	17
3.1.3. Pipeline de Trabajo e Integración de Recursos	17
3.2. Lenguaje de Programación: C#	18
3.2.1. Justificación Técnica y Arquitectura de Software	19
3.2.2. Rol en el Proyecto y Mecanismos Avanzados	19
3.2.3. Pipeline de Trabajo (Flujo de Desarrollo)	20
3.3. Entorno de Desarrollo Integrado (IDE): Visual Studio	21
3.3.1. Justificación Técnica y Elección del IDE	21
3.3.2. Rol en el Proyecto	21
3.3.3. Pipeline de Trabajo (Flujo de Integración)	22
3.4. Cliente de Control de Versiones: GitHub Desktop	22

3.5.	Soporte y Asistencia en el Desarrollo: Antigravity	23
3.6.	Herramientas de Diseño Gráfico, Animación y Audio	24
3.7.	Herramientas de Gestión, Comunicación y Planificación	25
3.8.	Metodología de Desarrollo Ágil Adaptada	26
3.9.	Control de Progreso y Revisiones Iterativas	26
3.10.	Diseño Iterativo y Greyboxing	27
3.11.	Planificación Temporal (Diagrama de Gantt)	27
4.	Descripción de los Juegos	29
4.1.	Historia de Reset	29
4.1.1.	Concepto y contexto narrativo	29
4.1.2.	Historia del prototipo jugable	30
4.2.	Historia, Ambientación y Contexto de los Niveles	31
4.2.1.	Mundo 1: Entorno de Supervivencia Cenital (“The Last Reset”)	31
4.2.2.	Mundo 2: Entorno de Plataformas Vectoriales (“Super Reset Bros”)	35
4.3.	Mecánicas y Dinámicas de Juego	37
4.3.1.	Mundo 1: Supervivencia Cenital (“The Last Reset”)	37
4.3.2.	Mundo 2: Plataformas Vectoriales (“Super Reset Bros”)	39
5.	Descripción Informática	43
5.1.	Arquitectura del Sistema y Persistencia Global	43
5.1.1.	Sistema de Persistencia y Guardado de Datos (SaveManager.cs)	43
5.1.2.	Sistema de Diálogos y Eventos Narrativos	46
5.1.3.	Iluminación Dinámica (FlickerLight2D.cs)	48
5.2.	Mundo 1: Mecánicas Cenitales y Algoritmos de Inteligencia Artificial	48
5.2.1.	Navegación Táctica por Direccionamiento de Contexto (Context Steering)	48
5.2.2.	Comportamiento Físico y Movimiento Orgánico de los Zombies	50
5.2.3.	El Algoritmo de Huida Táctica de los Enemigos a Distancia (Nivel 2)	51
5.2.4.	Sistema Anti-atasco por Teleportación Recuperativa	52
5.2.5.	Director de Drops y Ajuste de Dificultad Dinámico (DDA)	52
5.3.	Mundo 2: Físicas de Plataformas e Interacción Aérea	53
5.3.1.	Ajustes de Física Fina y Mecánicas de Game Feel	53
5.3.2.	Mecánicas de Parkour y Cristales de Recarga de Dash	54
5.3.3.	Zonas de Alteración Gravitatoria y Plataformas Inestables	55
5.3.4.	Sistema de Cámara Predictiva en Zonas	55
6.	Validación	58

7. Conclusiones y trabajos futuros	60
7.1. Grado de Cumplimiento de los Objetivos	60
7.2. Trabajo Futuro y Líneas de Ampliación	62
7.3. Conclusiones Personales	63
Bibliografía	65
Apéndices	67
A. Este es el primer apéndice	69
A.1. Ejemplo de funcionamiento de LaTeX	69
A.1.1. Tablas y figuras	69
A.2. Segunda sección	72
A.2.1. Código	74
A.2.2. Comentarios	75

Índice de tablas

5.1. Matriz de pesos probabilísticos del Director de Drops en base al estado del jugador.	52
A.1. Título de la tabla.	69
A.2. Tabla rotada. Factor groupings for the Mooshak questionnaire. . .	70
A.3. Tabla con “multicolumnas” y “multifilas”.	71

Índice de figuras

1.1. Títulos pioneros orientados a la superación de puntuaciones en la época arcade.	5
1.2. Títulos pioneros orientados a la superación de puntuaciones en la época arcade.	6
1.3. The Legend of Zelda: A Link to the Past (1991).	9
1.4. Títulos pixel art actuales de survival horror	9
1.5. Super Mario World (1991).	10
1.6. Títulos pixel art actuales de plataformas	11
3.1. Logotipo de Unity6.	16
3.2. Logotipo de C#.	19
3.3. Logotipo de Visual Studio.	21
3.4. Logotipo de GitHub Desktop.	23
3.5. Logotipo de Google Antigravity.	24
3.6. Herramientas de diseño y audio.	25
3.7. Logotipo de Microsoft Teams.	26
3.8. Diagrama de Gantt durante el desarrollo de Reset.	28
4.1. Inspiración del menú de <i>The Last Reset</i>	32
4.2. Inspiraciones de <i>The Last of Us</i> y <i>Resident Evil</i>	32
4.3. Vista general del Nivel 1 de <i>The Last Reset</i>	33
4.4. Nivel 2 de <i>The Last Reset</i>	34
4.5. Inspiración de <i>Super Reset Bros</i>	35
4.6. Inicio del Nivel 1 de <i>Super Reset Bros</i>	36
4.7. Inicio del Nivel 2 de <i>Super Reset Bros</i>	37
4.8. Controles globales de <i>The Last Reset</i>	39
4.9. Controles del Nivel 1 de <i>Super Reset Bros</i>	40
4.10. Controles del Nivel 2 de <i>Super Reset Bros</i>	41
A.1. Logo de la Universidad.	71

- A.2. Ejemplo con varias figuras. Demostración visual del teorema de Pitágoras. En (a) tenemos un triángulo rectángulo con hipotenusa c y catetos a y b . En (b) se muestra tres copias escaladas del mismo triángulo. El verde se ha escalado por a , el rojo/rosa por b , y el azul por c . En (c) se juntan los triángulos de (b) para formar un rectángulo cuya base es c^2 , pero también $a^2 + b^2$. Por tanto, $a^2 + b^2 = c^2$ 72

Índice de códigos

A.1. Título del algoritmo/código.	74
A.2. Endpoint REST para recuperar un libro por su id	74

1

Introducción

El presente Trabajo de Fin de Grado documenta el proceso de diseño y desarrollo de *Reset*, un videojuego en 2.5D de estética Pixel Art que explora y aúna diferentes géneros clásicos bajo una misma experiencia narrativa. En este primer capítulo se establecen las bases conceptuales del proyecto, exponiendo el contexto actual de la industria y las motivaciones principales que impulsan el desarrollo; analizando el estado del arte de los géneros y correspondientes subgéneros implementados y, finalmente, se detallan los objetivos generales y específicos que guían este trabajo.

1.0.1. Contexto del Proyecto

El proyecto *Reset* nace de la voluntad de integrar y sintetizar el conjunto de competencias y conocimientos técnicos adquiridos a lo largo del Grado Universitario. Desde una perspectiva de ingeniería del software y diseño de interacciones, el núcleo conceptual del juego reside en la **unificación de géneros discordantes dentro de una misma arquitectura de software**.

La motivación principal de este proyecto radica en el deseo de no limitar un trabajo de esta relevancia a un único ámbito del diseño y desarrollo de videojuegos. Por el contrario, se persigue un desarrollo global y transversal que permita plasmar las competencias y conocimientos adquiridos a lo largo del grado en todas sus dimensiones. Esta meta de carácter académico se unifica con una motivación de índole personal: concebir una obra cuyo trasfondo rinda homenaje a los títulos de referencia que han marcado la trayectoria del autor como jugador y que, en última instancia, despertaron su vocación por esta disciplina. En definitiva, el

proyecto busca establecer una sinergia entre el rigor técnico y conceptual exigido en el ámbito universitario y el tributo creativo a las obras que inspiraron el inicio de esta formación.

Asimismo, dado que el perfil profesional del autor no se limita exclusivamente al desarrollo técnico, sino que abarca un gran interés por áreas como el diseño visual y jugable o la interfaz y experiencia de usuario (UI/UX) y la animación, se optó por abordar el proyecto desde una perspectiva multidisciplinar. De este modo, la implementación de la arquitectura de software y los sistemas de programación se complementa de forma equilibrada con un riguroso trabajo en el apartado visual y de interacción, buscando un acabado final pulido que ponga en valor ambas facetas de la creación de videojuegos.

Por ello, en lugar de desarrollar un videojuego limitado a un único bucle de juego básico (*core gameplay loop*), *Reset* se diseña como una plataforma experimental que transiciona dinámicamente entre la perspectiva cenital (*Top-Down*) orientada a la acción y la supervivencia táctica, y la perspectiva lateral (*Side-Scroller*) orientada al plataformeo clásico y de precisión. Este enfoque plantea un desafío técnico directo: la creación de un sistema modular unificado capaz de gestionar físicas, inventarios y estados de juego completamente heterogéneos entre mundos, garantizando una experiencia de usuario fluida y coherente.

1.0.2. El panorama actual y la barrera de entrada a los géneros

La industria del videojuego ha evolucionado exponencialmente en las últimas décadas, diversificándose en una inmensa cantidad de géneros y subgéneros. Si bien esta variedad enriquece el medio, también genera un efecto de 'zonas de confort' entre los jugadores habituales, quienes tienden a consumir reiteradamente un mismo tipo de género (como los shooters, los juegos de rol o los títulos deportivos).

A este fenómeno se suma la propia evolución del desarrollo y la aparición de nuevos estudios de forma creciente año tras año, lo que ha provocado un incremento masivo en la publicación anual de títulos en comparación a periodos anteriores. Esta sobresaturación del mercado refuerza el aislamiento del jugador; al disponer de una oferta prácticamente inabarcable de su género favorito, el usuario tiende a especializarse. Inevitablemente, acaba convirtiéndose en un jugador de "nicho" que dedica todo su tiempo de ocio a iterar sobre las mecánicas que ya domina y disfruta, en lugar de abrir sus posibilidades a nuevas experiencias fuera de esa zona de confort o a recordar dinámicas y géneros que anteriormente disfrutaba.

Es en este contexto donde nace la motivación principal de *Reset*. El proyecto

se concibe con la ambición de abarcar al mayor número de perfiles de jugador posible, actuando como un puente estratégico. Para los jugadores de nicho, el título utiliza sus géneros predilectos como gancho inicial para captar su atención y, progresivamente, animarles a transicionar hacia mecánicas desconocidas.

Simultáneamente, *Reset* sirve como puerta de entrada para usuarios con poca experiencia, ofreciéndoles una cata interactiva donde pueden experimentar con diversos géneros y subgéneros. Para lograr este acercamiento bidireccional, el proyecto se apoya en la narrativa como ancla emocional, logrando que los jugadores menos experimentados empaticen con la historia y mantengan su atención mientras descubren sus propios gustos sin la fricción de tener que adquirir y aprender a jugar múltiples títulos independientes.

1.0.3. El reto del diseño: Cohesión frente a fragmentación

Al plantear un videojuego que combine múltiples géneros, el mayor riesgo en la fase de diseño era caer en la creación de una simple colección de minijuegos inconexos. A nivel personal y profesional, el objetivo primordial era deconstruir esa estructura fragmentada para construir una experiencia verdaderamente lineal y cohesiva.

Para lograr que los jugadores no sintieran estos saltos de género como una ruptura del *game feel*, era imperativo diseñar un hilo conductor lo suficientemente robusto. La solución adoptada fue dotar al proyecto de una narrativa profunda y emocional que justificara, de manera orgánica dentro del propio universo del juego, cada cambio de perspectiva, física y control.

De la misma forma, esta narrativa serviría como gancho para los jugadores principiantes. Al tratarse de mecánicas, perspectivas y ambientaciones completamente diferentes en cada nivel, el usuario podría perderse o frustrarse al encontrarse con un género que quizás no logre captar toda su atención. Es en ese momento donde la narrativa rescata la percepción de este tipo de jugador, ayudándole a progresar con el fin general de completar una historia con la que previamente haya conectado.

1.0.4. El reto de jugabilidad: Imposibilidad frente a aburrimiento

Aglutinar a un público tan dispar —desde el jugador de nicho altamente experimentado hasta el usuario novato— plantea un desafío de diseño crítico: el balanceo de la dificultad. Basándose en la teoría de flujo, si un nivel es demasiado exigente, el jugador inexperto chocará contra un muro de imposibilidad y abandonará el juego por frustración; por el contrario, si el reto es excesivamente

simple, el jugador veterano caerá en el aburrimiento.

Para resolver esta dicotomía en *Reset*, se ha optado por un diseño de niveles estructurado en capas. La ruta principal de cada mundo está diseñada para ser un reto accesible con su dedicada curva de aprendizaje, donde las métricas de física y la agresividad de los enemigos permiten un avance fluido con un amplio margen de error. Esto garantiza que cualquier jugador pueda disfrutar de la transición de géneros y avanzar en la historia. Sin embargo, para satisfacer a los jugadores más experimentados que buscan exprimir al máximo el rendimiento y las mecánicas, se integran rutas secundarias y desafíos opcionales. Estas bifurcaciones elevan drásticamente el nivel de exigencia, premiando la maestría y la precisión sin penalizar el progreso general del resto de usuarios.

1.1. Estado del arte

El diseño y desarrollo de un videojuego multigénero requiere una comprensión profunda de las convenciones, mecánicas y la evolución histórica de los estilos que busca unificar. Para contextualizar adecuadamente el enfoque y las decisiones de diseño tomadas en *Reset*, se analiza el contexto de evolución de los géneros de videojuegos, los cuales con el paso del tiempo han derivado en una enorme diversificación, así como el estado del arte de las dos grandes corrientes lúdicas que conforman el núcleo de este prototipo funcional (*Vertical Slice*): la aventura de acción con tintes de supervivencia en perspectiva cenital (*Top-Down*) y el género de plataformas bidimensional (*Side-Scrolling*), abarcando tanto su vertiente más clásica como el plataformas de precisión moderno.

A través de este análisis, se explorarán las obras referentes que sentaron las bases fundacionales de los videojuegos hasta derivar en ambos paradigmas mencionados, cómo han evolucionado sus mecánicas principales a lo largo de las décadas y de qué manera sus filosofías de diseño influyen en la actualidad. Comprender este recorrido histórico es un paso fundamental para justificar las resoluciones arquitectónicas y jugables implementadas durante el desarrollo de este proyecto. De este modo, se explican la integración de mecánicas dispares entre niveles, unificadas bajo un mismo motor y una estética cohesionada, representando un estudio técnico y práctico de la propia historia del medio interactivo.

1.1.1. El origen y la diversificación de los géneros en el videojuego

En los comienzos de la industria de los videojuegos, las limitaciones tecnológicas dictaban de forma absoluta el diseño de las experiencias jugables. Durante la década de los setenta y principios de los ochenta, el nacimiento de los videojue-

gos estuvo íntimamente ligado a los salones recreativos. En este contexto emergió el género Arcade, caracterizado por mecánicas directas, partidas de muy corta duración y una curva de dificultad extremadamente pronunciada. Su diseño no buscaba la inmersión a largo plazo, sino incentivar al jugador a intentar una y otra vez superar una puntuación o nivel con el fin de hacerle gastar más monedas en la misma máquina recreativa. Títulos como *Space Invaders* (1978) o *Pac-Man* (1980) (ver Figura 1.1a) sentaron las bases de la interacción digital, pero limitaban su alcance a la superación de puntuaciones en pantallas estáticas o con un desplazamiento bidimensional muy rudimentario.

Sin embargo, la llegada y democratización de las consolas domésticas de 8 y 16 bits como la NES de Nintendo (ver Figura 1.1b) o la Mega Drive de SEGA, supuso un cambio de paradigma radical. Aunque la transición al salón del hogar se inició previamente con el lanzamiento de la *Magnavox Odyssey* (1972), esta primera etapa aún heredaba el modelo de diseño y consumo efímero del entorno arcade. Fue con el desembarco de las plataformas japonesas cuando los desarrolladores se liberaron de la necesidad de diseñar en torno a la monetización rápida, permitiendo concebir partidas de larga duración. La introducción de sistemas de guardado en la memoria —inicialmente mediante contraseñas y posteriormente a través de pilas internas en los propios cartuchos— fue el catalizador definitivo para la diversificación y explosión de los géneros. Al dotar a los juegos de memoria, se abría la puerta al progreso continuado y a la narrativa extensa, dando lugar a títulos con una línea de progresión clara, un inicio definido y un desenlace.



(a) Pantallas de juego de *Space Invaders* (1978) y *Pac-Man* (1980).



(b) Nintendo NES original (1985).

Figura 1.1: Títulos pioneros orientados a la superación de puntuaciones en la época arcade.

Es en esta etapa histórica es donde se consolidan las grandes corrientes lúdicas que cimentaron las bases de la industria actual y que sirven de referente directo para este proyecto:

- **Juegos de Acción y Plataformas:** Evolucionaron desde el formato de salto en una pantalla estática hacia el desplazamiento lateral continuo (*Side-*

Scrolling). Se priorizó la habilidad rítmica, el control de la inercia y la superación de obstáculos físicos, convirtiéndose en el género estandarte de las consolas de 8 y 16 bits gracias a referentes como *Super Mario Bros.* (1985) o *Sonic the Hedgehog* (1991) (ver Figura 1.2). El primero supuso uno de los grandes puntos de inflexión en la concepción lúdica de los ochenta, ya que fue de los primeros títulos en trasladar el estilo arcade a la unificación narrativa. Con ello, dotó al juego de un ciclo cohesivo mediante una travesía estructurada a lo largo de diferentes reinos, donde cada nivel respondía a una lógica espacial y argumental unificada. Este salto de creatividad impulsó el desarrollo hacia nuevos enfoques, abriendo un abanico de posibilidades que cimentó las bases del diseño contemporáneo.



(a) Super Mario Bros (1985).



(b) Sonic The Hedgehog (1991).

Figura 1.2: Títulos pioneros orientados a la superación de puntuaciones en la época arcade.

- **Aventuras y Juegos de Rol (RPG):** Los primeros juegos de aventura nacieron como experiencias basadas en texto, siendo *Colossal Cave Adventure* (1976) el pionero en fundar el género mediante la resolución de acertijos en pantallas interactivas, lo que más tarde derivó en las aventuras gráficas de los 80 y 90. En paralelo, los RPG digitalizaron los sistemas de los juegos de rol de mesa. Ambos géneros rompieron con la linealidad de los niveles estáticos para ofrecer mundos interconectados enfocados en la exploración y la narrativa; para lograrlo, los RPG de la época estandarizaron la perspectiva cenital (*Top-Down*) como la herramienta visual óptima para gestionar el espacio y mejorar la experiencia del usuario.

1.1.2. La historia reciente y la diversificación de subgéneros

Con el paso de los años, la evolución tecnológica y la creatividad que los diseñadores impregnaban en cada título —en producciones donde las barreras

creativas parecían disolverse— provocaron la ramificación de los géneros de videojuegos en un sinfín de subgéneros. Estas variantes nacían tanto de cambios radicales en la jugabilidad dentro de un mismo género, como de configuraciones concretas de la cámara o modificaciones sutiles de sus mecánicas básicas.

En la última década, este fenómeno de hiperespecialización se ha visto acelerado exponencialmente por la democratización de motores gráficos comerciales y herramientas de desarrollo accesibles. Esta apertura tecnológica propició el denominado "boom indie" (desarrollo independiente), un ecosistema donde estudios de menor tamaño, liberados de las rígidas exigencias comerciales de las grandes superproducciones (mercado AAA), comenzaron a deconstruir y experimentar con las convenciones clásicas del medio.

Como resultado, la industria contemporánea se caracteriza profundamente por la hibridación mecánica. Las fronteras estrictas que durante los años noventa separaban la acción, la estrategia o la aventura se han difuminado para dar paso a la integración de sistemas complejos. Conceptos modernos como el *Roguelite* (generación procedimental con progresión permanente) o el *Soulslike* (énfasis en el castigo por el error, la gestión meticulosa de la resistencia y la narrativa ambiental) han permeado en la inmensa mayoría de las categorías tradicionales. Los jugadores actuales ya no buscan experiencias unidimensionales, sino propuestas que combinen capas de diferentes géneros para generar nuevas dinámicas de juego.

Dentro de este panorama reciente, los dos géneros que vertebran este proyecto han sufrido redefiniciones fundamentales:

- Por un lado, las aventuras en perspectiva cenital (*Top-Down*) han trascendido la simple exploración lineal o el combate básico (*Hack and Slash*). En la actualidad, estos entornos bidimensionales se utilizan frecuentemente para enmarcar experiencias de supervivencia (*Survival Horror*), donde el control de una cámara limitada se aprovecha para generar puntos ciegos y tensión psicológica. La jugabilidad contemporánea en estos subgéneros exige la gestión táctica de recursos muy escasos y el enfrentamiento contra inteligencias artificiales que obligan al jugador a medir cada enfrentamiento.
- Por otro lado, el género de plataformas ha vivido un renacimiento técnico y de diseño a través de su variante más extrema: el plataformas de precisión (*Precision Platformer*). Obras referentes de los últimos diez años han optado por eliminar por completo las penalizaciones tradicionales (como el sistema de vidas limitadas o los tiempos de carga tras morir), apostando por la reaparición instantánea. Este cambio de paradigma enfoca la experiencia en el ensayo y error a gran velocidad, exigiendo al usuario un dominio casi coreográfico de mecánicas de movilidad avanzada, como el salto rebotado (*Wall Jump*), el deslizamiento y el impulso aéreo (*Dash*).

Paralelamente a esta evolución mecánica, la historia reciente del videojuego

ha consolidado el uso de la estética *Pixel Art*. Lejos de ser una limitación impuesta por el hardware como en sus orígenes, en la actualidad se trata de una decisión artística y nostálgica deliberada que busca evocar la época dorada de las consolas de 16 bits. *Reset* se enmarca exactamente en este contexto de diseño moderno: utiliza el estilo visual retro para apelar a la memoria histórica del jugador, pero aplica una arquitectura de hibridación de subgéneros y mejoras de calidad de vida (*Quality of Life*) para ofrecer una experiencia de juego adaptada a los exigentes estándares de fluidez (*Game Feel*) contemporáneos.

Esas ramificaciones se han ido completando hasta crear un árbol de dimensiones y complejidad casi incomprensibles para la mayoría de jugadores casuales. Ese es uno de los pilares que se tratan en *Reset*, que recoge esta herencia histórica y asume la complejidad técnica de difuminar dichas fronteras, integrando en una sola arquitectura las diversas mecánicas y configuraciones jugables analizadas.

1.1.3. Justificación de la arquitectura de diseño en RE-SET: Modelos de hibridación por paradigmas

El desarrollo de un proyecto de software interactivo de forma unipersonal exige una planificación rigurosa que optimice los recursos técnicos sin sacrificar la profundidad de la experiencia de usuario. En este contexto, *Reset* se estructura formalmente a través de dos paradigmas jugables bien diferenciados, distribuidos en mundos independientes. Esta segmentación no solo divide la carga del diseño de niveles (*Level Design*), sino que sirve para demostrar cómo filosofías de juego propias de grandes producciones contemporáneas en tres dimensiones —como la tensión de *Resident Evil* o las dinámicas de aventura de *The Last of Us*— pueden ser deconstruidas y trasladadas con éxito a un entorno bidimensional optimizado bajo la perspectiva *Top-Down* y el formato de desplazamiento lateral.

Paradigma 1: La Aventura Cenital y la adaptación del Survival Horror

El primer bloque del proyecto articula la convergencia entre la exploración clásica y la tensión psicológica, estructurando la experiencia en dos subgéneros claramente diferenciados que representan distintas evoluciones de la vista cenital.

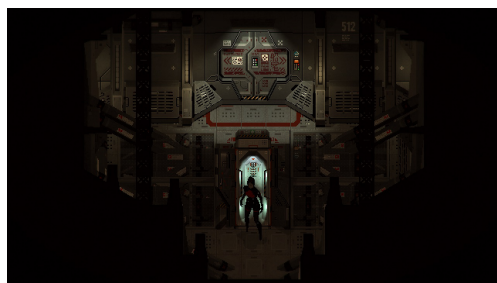
Para el primer nivel, el diseño se decanta hacia la acción-aventura con un enfoque directo en el combate (*Hack and Slash*). La base del sistema de enfrentamiento cuerpo a cuerpo y el tránsito espacial se fundamenta en los estándares establecidos por títulos como *The Legend of Zelda: A Link to the Past* (1991) (ver Figura 1.3), que demostró la viabilidad de la interacción táctica en espacios bidimensionales. En esta fase, la jugabilidad prioriza la agilidad en la exploración de las diversas zonas y el combate directo contra los enemigos, sirviendo como una primera toma de contacto centrada en el dinamismo y la acción mecánica.



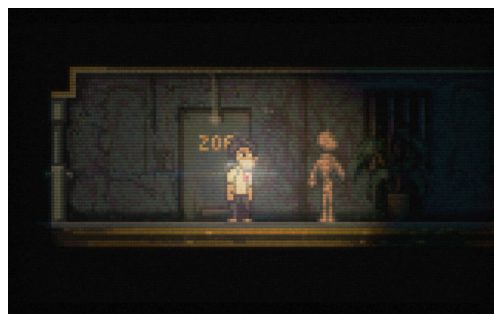
Figura 1.3: The Legend of Zelda: A Link to the Past (1991).

Sin embargo, en el segundo nivel de este mundo, *Reset* altera la naturaleza ágil de la aventura clásica al inyectarle las mecánicas sistémicas del *Survival Horror* puro. La dinámica cambia por completo al introducir el combate a distancia, obligando al jugador a explorar exhaustivamente los escenarios en busca de notas, objetos clave y contraseñas para poder resolver los puzles que bloquean el avance.

Para lograrlo, se toma como referencia la atmósfera opresiva y la escasez de recursos de obras referentes de la supervivencia como *The Last of Us*, combinándolo con la implementación técnica de un inventario matricial limitado por cuadrículas característico de la saga *Resident Evil*. Esta decisión de diseño traslada la toma de decisiones estratégicas al jugador, quien debe evaluar el coste de cada enfrentamiento y decidir si emplear su valiosa munición o recurrir a la evasión. Casos de estudio modernos como *Signalis* (2022) o el enfoque bidimensional de *Lone Survivor* (2012) (ver Figura 1.4) validan esta metodología, demostrando que la vulnerabilidad del avatar, la restricción de visibilidad y la atmósfera opresiva no dependen de una tecnología tridimensional, sino de la coherencia en la aplicación de las reglas lúdicas dentro de un entorno *Pixel Art*.



(a) *Signalis* (2022).



(b) *Lone Survivor* (2012).

Figura 1.4: Títulos pixel art actuales de survival horror

Paradigma 2: Del Plataformas Clásico a la Hiperespecialización de Precisión

El segundo entorno del juego aborda una transición pedagógica y técnica sobre la evolución de la movilidad del personaje, dividiendo la experiencia en dos niveles de dificultad orgánica:

- **El Plataformas Clásico:** Configurado como un entorno introductorio permisivo con el usuario, utiliza como marco de estudio obras como *Super Mario World* (1990) (ver Figura 1.5). En esta fase, el diseño del código prioriza el control intuitivo de la inercia, la parábola física del salto y la eliminación de amenazas mediante la colisión vertical del avatar sobre los enemigos, asentando una base accesible de control del espacio.

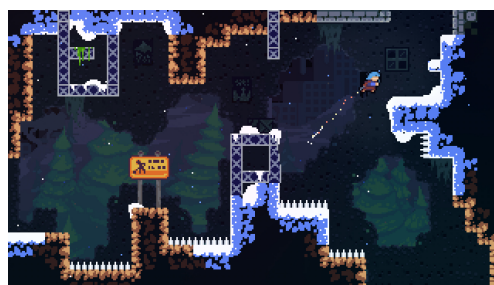


Figura 1.5: Super Mario World (1991).

- **El Plataformas de Precisión:** Concebido como el pico de dificultad técnica del proyecto, este nivel evoluciona hacia el formato del *Precision Platformer*. Se toman como pilares fundamentales de diseño *Super Meat Boy* (2010) (ver Figura 1.6a), por su aproximación al bucle rápido de ensayo y error frenético, y *Celeste* (2018) (ver Figura 1.6b), del cual se realiza una traslación directa de mecánicas avanzadas. El motor de *Reset* replica de este último el impulso aéreo en las diferentes direcciones (*Dash*), el deslizamiento y agarre a superficies verticales (*Wall Slide*) con su correspondiente impulso (*Wall Jump*) y el impulso extra en el aire en forma de salto (*Double Jump*). Estructuralmente, destaca la fragmentación del escenario en micro-retos independientes contenidos en zonas marcadas. Este enfoque técnico elimina los sistemas arcaicos de penalización por vidas limitadas, reenfocando la frustración lúdica del usuario hacia la ejecución coreográfica y precisa de los controles.



(a) Super Meat Boy (2010).



(b) Celeste (2018).

Figura 1.6: Títulos pixel art actuales de plataformas

2

Objetivos

En base al análisis realizado en el capítulo anterior, se establecen las metas fundamentales de este Trabajo de Fin de Grado. El propósito principal de este proyecto es dar respuesta a la sobresaturación y fragmentación del mercado mediante la creación de una obra que actúe como puente entre diferentes perfiles de usuario y así impulsar además la llegada de nuevos jugadores.

Por tanto, el **objetivo general** de este proyecto es diseñar y desarrollar un prototipo jugable (*Vertical Slice*) de **Reset**, un videojuego en 2.5D con estética *Pixel Art* para PC que unifica múltiples paradigmas lúdicos bajo un mismo hilo conductor, sirviendo como demostración técnica y de diseño de las competencias adquiridas durante el grado.

Para alcanzar esta meta global, se definen los siguientes **objetivos específicos**:

- **Elaborar un Documento de Diseño de Juego (GDD) exhaustivo:** Definir y documentar de forma estructurada todas las mecánicas, sistemas de control, enemigos y arquitecturas de niveles que compondrán el prototipo, garantizando la viabilidad y versatilidad de la idea original.
- **Desarrollar una narrativa cohesiva como hilo conductor:** Crear e implementar una historia que justifique de manera orgánica los cambios drásticos de cámara, físicas y géneros entre los diferentes mundos, evitando que el jugador perciba la experiencia como una colección de minijuegos inconexos.

- **Implementar un diseño de niveles estructurado en capas:** Aplicar la teoría de la fluidez para balancear la dificultad, construyendo una ruta principal accesible con su debida curva de aprendizaje para jugadores noveles y diseñando retos secundarios de alta exigencia para retener el interés de los jugadores veteranos.
- **Programar un motor de físicas y control en 2D adaptado al género de plataformas:** Diseñar y calibrar las mecánicas de movimiento dinámico del jugador, integrando algoritmos de *Game Feel* para asegurar una experiencia reactiva, precisa y satisfactoria.
- **Desarrollar una arquitectura escalable de Inteligencia Artificial:** Estructurar los agentes enemigos mediante principios de programación orientada a objetos (herencia y polimorfismo) e implementar algoritmos de navegación y de toma de decisiones reactiva para garantizar comportamientos orgánicos y adaptativos.
- **Implementar un sistema robusto de persistencia de datos y gestión de estado:** Crear una arquitectura de guardado y carga centralizada que serialice en formato JSON el estado de todas las variables, favoreciendo la continuidad de la experiencia de forma compacta y segura.
- **Integrar sistemas mecánicos específicos para cada género:** Diseñar e implementar las lógicas particulares de cada entorno para conformar una experiencia de juego sólida, inmersiva y coherente en su conjunto.

3

Tecnologías, Herramientas y Metodologías

En esta sección se detallan todas las tecnologías utilizadas a lo largo del desarrollo del proyecto, desde lenguajes y sistemas internos del motor de desarrollo hasta software concreto de diseño audiovisual y de gestión del proyecto. También se tratan las metodologías utilizadas en los meses de desarrollo tanto para la organización de tareas como de comunicación y supervisión con el tutor.

3.1. Motor de Videojuegos: Unity 6

Para el desarrollo del prototipo se ha seleccionado la última iteración del motor de videojuegos de Unity Technologies, conocida comercialmente como **Unity 6** (versión de compilación específica 6000.3.8f1) (ver Figura 3.1).



Figura 3.1: Logotipo de Unity6.

3.1.1. Justificación Técnica y Elección del Motor

La elección de Unity 6 frente a otras alternativas del mercado se fundamenta en criterios de eficiencia de desarrollo, arquitectura interna y viabilidad para proyectos independientes:

- **Especialización en entornos bidimensionales:** A diferencia de otros motores optimizados prioritariamente para el renderizado fotorrealista en tres dimensiones, Unity 6 integra un ecosistema nativo avanzado para el desarrollo 2D y 2.5D. Herramientas como el sistema de *Tilemaps* o el empaquetador de gráficos optimizan la creación de escenarios modulares y la gestión de recursos visuales sin sobrecargar el rendimiento del sistema.
- **Ecosistema y documentación técnica:** La API de Unity destaca por ser una de las más documentadas y estandarizadas de la industria. Disponer de un marco de soporte tan amplio mitiga los riesgos técnicos durante el desarrollo, facilitando la resolución de problemas lógicos y la implementación de patrones de diseño de software extendidos.
- **Modelo de licenciamiento:** El acceso a la licencia *Unity Personal* permite el uso íntegro de las capacidades del motor de forma gratuita para proyectos académicos o de investigación sin presupuesto inicial, eliminando barreras económicas y garantizando la viabilidad del proyecto.
- **Curva de experiencia previa:** El uso de este motor de videojuegos a lo largo de la formación académica del grado garantiza un dominio técnico avanzado de su interfaz y arquitectura. Optar por una tecnología familiar reduce drásticamente el tiempo invertido en la fase de adaptación y aprendizaje básico, lo que se traduce en una mayor eficiencia operativa, soltura en la resolución de problemas y optimización de los plazos de entrega del proyecto.

3.1.2. Rol en el Proyecto

Unity 6 opera como el núcleo central del videojuego, coordinando los subsistemas principales que componen la experiencia interactiva y garantizando la cohesión entre la lógica de software y la representación multimedia. Su rol se distribuye en las siguientes áreas fundamentales:

- **Subsistema de Físicas Dinámicas (Box2D):** El motor gestiona de manera nativa la simulación de fuerzas, gravedad y colisiones aplicadas sobre los elementos mecánicos del juego. Este motor bidimensional se emplea tanto para calcular el movimiento preciso del personaje en entornos de plataformas como para realizar comprobaciones de proximidad y detecciones espaciales en tiempo real entre las distintas entidades del mundo virtual.
- **Canalización de Renderizado (Universal Render Pipeline - URP):** Se ha utilizado la tubería de renderizado ligero URP configurada específicamente para entornos bidimensionales. Esta tecnología permite procesar de manera eficiente la estética visual del juego, gestionando el comportamiento de las cámaras, la ordenación de las capas de renderizado (*Sorting Layers*) y el procesamiento de imágenes con un bajo consumo de recursos de hardware.
- **Controlador de Animaciones (Mecanim):** El sistema nativo de animación gestiona los ciclos visuales de las entidades mediante máquinas de estado jerárquicas. Este componente traduce las variables lógicas controladas por código en transiciones visuales fluidas, aislando la lógica pura del juego de la representación gráfica del personaje.
- **Administración del Ciclo de Vida y Gestión de Escenas:** El motor actúa como el gestor de memoria principal del proyecto, controlando la carga y descarga asíncrona de los diferentes niveles, mundos y menús interactivos. Esta característica es fundamental para asegurar la persistencia de los datos de control y garantizar transiciones fluidas entre los distintos estados del juego sin provocar caídas en el rendimiento.
- **Sistema de Interfaz de Usuario (UI):** Unity coordina la capa de presentación gráfica superpuesta que comunica información crítica al jugador (como indicadores de estado, inventarios o menús de navegación). El motor gestiona de forma nativa la escalabilidad de estos elementos según la resolución de la pantalla y procesa los eventos de interacción del usuario de manera aislada al bucle lógico del juego.

3.1.3. Pipeline de Trabajo e Integración de Recursos

El flujo de desarrollo dentro de Unity 6 se ha estructurado como un proceso sistemático para asimilar la naturaleza multigénero del proyecto. El motor actúa

como un entorno unificado donde confluyen el diseño modular de niveles y la arquitectura de software a través de tres fases clave:

1. **Procesamiento y Estructuración de Activos Visuales:** Los recursos gráficos bidimensionales importados se configuran mediante perfiles específicos sin compresión para preservar la fidelidad del estilo artístico. El uso del *Sprite Editor* integrado permite segmentar las hojas de recursos y definir de manera precisa los puntos de pivote físicos de las entidades, garantizando que las detecciones de suelo y las alineaciones espaciales sean coherentes con la lógica de control.
2. **Construcción Modular de Escenarios (*Tilemaps*):** La arquitectura de los niveles se diseña de forma eficiente mediante el sistema nativo de rejillas y paletas de celdas. Esta metodología permite pintar los entornos de manera modular y automatizar la generación de colisiones físicas mediante componentes como *Tilemap Collider 2D* y *Composite Collider 2D*. Este flujo agiliza el diseño de los escenarios de los diferentes mundos y optimiza el rendimiento del motor al fusionar múltiples geometrías de colisión individuales en una única malla unificada.
3. **Vinculación Lógica y Parametrización en el Inspector:** El nexo entre el código en C# y el entorno visual se realiza a través de la interfaz del Inspector de Unity. Al exponer y serializar variables críticas de los scripts (como fuerzas de salto, velocidades de movimiento o tiempos de reutilización), el motor permite modificar estos atributos en tiempo de ejecución. Esta característica resulta fundamental para calibrar y equilibrar las distintas mecánicas y físicas exigidas por cada género de videojuego sin necesidad de reescribir ni recompilar el código fuente constantemente.

3.2. Lenguaje de Programación: C#

Toda la lógica de juego, la simulación física personalizada, la inteligencia artificial de las entidades y el sistema de guardado y carga han sido implementados en C# (C-Sharp) (ver Figura 1.5). C# es un lenguaje de programación multiparadigma moderno, fuertemente tipado y administrado por el entorno de desarrollo de Unity, actuando como el lenguaje vehicular para estructurar las mecánicas interactivas del videojuego.

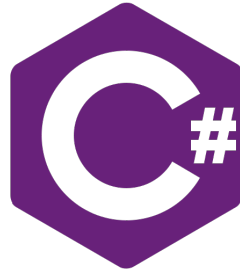


Figura 3.2: Logotipo de C#.

3.2.1. Justificación Técnica y Arquitectura de Software

La selección de C# responde a criterios de eficiencia en el tiempo de desarrollo, seguridad en la gestión de memoria dinámica y soporte arquitectónico para la creación de software modular:

- **Gestión automatizada de memoria (Garbage Collector):** A diferencia de lenguajes de bajo nivel como C++, C# opera sobre un entorno administrado que cuenta con un recolector de basura. Este componente libera automáticamente la memoria de los objetos dinámicos que dejan de ser referenciados por el juego, previniendo fugas de memoria (*memory leaks*) y garantizando la estabilidad de la aplicación sin exigir una gestión manual de punteros.
- **Paradigma Orientado a Objetos (POO):** El lenguaje proporciona un soporte robusto para la implementación de principios de diseño orientado a objetos como la herencia y el polimorfismo. Esto resulta fundamental para abordar la naturaleza de un proyecto que combina múltiples géneros, permitiendo estructurar plantillas de comportamiento genéricas para las entidades y extenderlas en componentes especializados, reduciendo la redundancia y facilitando el mantenimiento del código fuente.
- **Desacoplamiento modular y delegados:** C# facilita la creación de arquitecturas desacopladas mediante el uso de eventos y delegados. Esto permite separar la lógica de detección física del jugador de la respuesta de los elementos interactivos del entorno (como cajas, interfaces o portales), logrando que los sistemas se comuniquen entre sí de manera limpia y sin dependencias directas entre componentes.

3.2.2. Rol en el Proyecto y Mecanismos Avanzados

El lenguaje se ha empleado para programar las reglas y comportamientos del videojuego, coordinando la lógica abstracta a través de los siguientes mecanismos

del entorno:

- **Gestión del Ciclo de Vida del Motor:** C# permite interactuar directamente con el bucle de ejecución del motor. De esta forma, el código coordina de manera ordenada la sincronización de las entradas del jugador, la actualización periódica de la lógica de los enemigos y el procesamiento de la simulación física de forma independiente a la tasa de fotogramas del dispositivo.
- **Ejecución Asíncrona (Corrutinas):** El lenguaje ofrece un mecanismo nativo para suspender la ejecución de una función y reanudarla en fotogramas posteriores. Esto es clave para programar comportamientos basados en intervalos de tiempo (como cadencias de disparo, tiempos de aturdimiento de los enemigos o transiciones visuales de fundido) sin necesidad de bloquear el hilo de ejecución principal ni ralentizar la tasa de refresco visual.
- **Serialización de Datos:** C# proporciona las herramientas necesarias para traducir estructuras de datos complejas cargadas en la memoria activa del juego a formatos de almacenamiento estandarizados. Esto permite guardar y recuperar de forma estructurada el estado del inventario, los coleccionables y el progreso del jugador en el almacenamiento físico del sistema operativo.

3.2.3. Pipeline de Trabajo (Flujo de Desarrollo)

El ciclo de ingeniería de software con C# sigue un proceso cerrado de implementación y verificación técnica dentro del entorno de desarrollo:

1. **Programación de Componentes:** Se desarrollan scripts lógicos que heredan de las clases base del motor, lo que permite acoplarlos directamente a los elementos visuales y físicos del entorno de desarrollo interactivo.
2. **Depuración y Trazabilidad de Errores:** El flujo de trabajo aprovecha la integración directa del código con las herramientas de diagnóstico del entorno de desarrollo integrado (IDE). Esto permite monitorizar el flujo de la lógica, examinar el estado de las variables lógicas en tiempo real e insertar puntos de interrupción (*breakpoints*) para localizar fallos lógicos durante las fases de prueba.

3.3. Entorno de Desarrollo Integrado (IDE): Visual Studio

Para la escritura, edición y depuración de todo el código fuente en C#, se ha seleccionado el entorno de desarrollo integrado **Microsoft Visual Studio** (en su edición *Community*) (ver Figura 3.3).

3.3.1. Justificación Técnica y Elección del IDE

La elección de Visual Studio frente a otros editores de código ligero (como Visual Studio Code o Notepad++) responde a criterios de robustez técnica e integración nativa con el motor de videojuegos:



Figura 3.3: Logotipo de Visual Studio.

- **Integración nativa con Unity:** Visual Studio cuenta con herramientas integradas específicas para Unity (*Game Development with Unity workload*). Esto permite la sincronización automática de la solución del proyecto, facilitando que el IDE reconozca instantáneamente todas las clases, métodos e interfaces propias de la API del motor.
- **IntelliSense y análisis de código estático:** El motor de autocompletado inteligente ayuda a reducir errores de sintaxis y tipografía en tiempo de escritura. Además, el análisis estático advierte sobre posibles problemas de rendimiento específicos de Unity (como el uso ineficiente de búsquedas de componentes repetitivas o llamadas de físicas incorrectas) antes de compilar el software.
- **Licencia comunitaria gratuita:** La versión *Community* ofrece todas las capacidades profesionales de depuración y refactorización de código sin coste para entornos educativos y desarrolladores independientes, garantizando la viabilidad de la herramienta dentro del proyecto.

3.3.2. Rol en el Proyecto

Visual Studio opera como el espacio de trabajo principal de ingeniería de software del videojuego, cumpliendo los siguientes roles:

- **Edición y Refactorización de Código:** Permite organizar la arquitectura de scripts y realizar refactorizaciones seguras (como renombrar clases o variables en múltiples archivos a la vez) sin romper las dependencias lógicas del juego.
- **Depuración activa (Debugging) mediante acoplamiento:** El IDE permite acoplar su proceso de ejecución al editor de Unity en tiempo real. Esto hace posible insertar puntos de interrupción (*breakpoints*) en líneas específicas de código. Cuando el juego se ejecuta y alcanza esa línea, la ejecución se congela, permitiendo al desarrollador examinar el valor exacto de las variables en memoria física (útil para diagnosticar fallos en la IA de los enemigos o en el comportamiento del selector de niveles).

3.3.3. Pipeline de Trabajo (Flujo de Integración)

El flujo de trabajo entre Visual Studio y Unity se realiza a través de un puente bidireccional automático:

1. **Apertura y sincronización:** Al hacer doble clic en cualquier script en la interfaz de Unity, el motor de videojuegos abre el archivo en Visual Studio, sincronizando las referencias del proyecto de C#.
2. **Compilación asíncrona:** Al guardar los cambios en Visual Studio, Unity detecta de manera asíncrona la actualización del archivo en disco y compila automáticamente las modificaciones, permitiendo probar los cambios en el editor de forma casi instantánea.

3.4. Cliente de Control de Versiones: GitHub Desktop

Para la administración del sistema de control de versiones Git y la sincronización con el repositorio remoto alojado en la nube, se ha seleccionado el cliente gráfico oficial **GitHub Desktop** (ver Figura 3.4). La elección de esta herramienta se justifica por la familiaridad y experiencia previa adquirida con su interfaz a lo largo del grado, lo que garantizó una curva de aprendizaje nula y una mayor agilidad en el desarrollo diario.



Figura 3.4: Logotipo de GitHub Desktop.

A pesar de tratarse de un proyecto individual, se ha implementado de forma rigurosa una metodología de trabajo basada en ramas (*branching*) para garantizar la estabilidad del software. En el desarrollo de videojuegos, las escenas de Unity (*.unity*) y los prefabs (*.prefab*) son archivos estructurados en formato YAML que se corrompen con facilidad y son extremadamente difíciles de fusionar ante cualquier conflicto.

El uso de ramas independientes ha actuado como un entorno de pruebas seguro (*sandbox*): cada nueva característica compleja (como la inteligencia artificial o el sistema de menús de pausa) se desarrolló en su propia rama dedicada. De este modo, si una implementación experimental fallaba o corrompía la escena de trabajo, bastaba con descartar esa rama sin poner en riesgo la compilación principal y estable del videojuego, la cual permanecía a salvo en la rama *main*. Además, este modelo facilita la trazabilidad del proyecto al asociar cada rama a una tarea específica del tablero de gestión.

3.5. Soporte y Asistencia en el Desarrollo: Antigravity

Como parte del conjunto de herramientas de desarrollo, se ha utilizado el asistente de inteligencia artificial *Antigravity* (desarrollado por el equipo de Google DeepMind) (ver Figura 3.5) para dar soporte en las tareas de programación y documentación técnica del proyecto. Su integración en el flujo de trabajo ha operado bajo la metodología de programación en pareja (*Pair Programming*), sirviendo de apoyo técnico en tres áreas fundamentales:

- **Depuración de Ciclos de Vida y Sincronía:** Asistencia en la resolución de condiciones de carrera en el orden de ejecución nativo de Unity.
- **Estructuración de Algoritmia Física y Matemática:** Soporte en el desarrollo y validación matemática de las ecuaciones de movimiento, tales como el cálculo de vectores o las parábolas físicas del personaje.

- **Auditoría de Arquitectura de Código:** Colaboración en la estructuración modular de los gestores globales del juego, facilitando la implementación de patrones de diseño limpios y el desacoplamiento de datos a través de *ScriptableObjects*.

De esta forma, la herramienta ha funcionado como un recurso de consulta y refactorización que ha permitido optimizar los tiempos de desarrollo, facilitando que el alcance del proyecto se viera proyectado exponencialmente en pro de una mejor experiencia de usuario.



Figura 3.5: Logotipo de Google Antigravity.

3.6. Herramientas de Diseño Gráfico, Animación y Audio

Para dar soporte a la creación de los recursos visuales, las interfaces y los efectos sonoros del videojuego, se han empleado diversas herramientas multimedia especializadas:

- **Adobe Photoshop:** Esta herramienta de edición y manipulación de gráficos rasterizados (*bitmaps*) de alta resolución se ha utilizado para el desarrollo de la identidad visual del proyecto principal y de los subentornos interactivos de cada nivel. Asimismo, se ha empleado en la composición técnica de las imágenes de fondo y en la creación, maquetación y dimensionado de los elementos constitutivos de la interfaz de usuario (UI), garantizando una canalización limpia de texturas hacia Unity 6. La elección de este software se fundamenta en la experiencia previa consolidada a lo largo de la formación académica. Esta soltura técnica ha permitido simplificar la creación de interfaces complejas y optimizar los tiempos de preproducción gráfica, asegurando un acabado estético profesional que facilita la posterior integración modular dentro del motor de videojuegos (ver Figura 3.6a).
- **Piskel:** Se trata de un editor de gráficos bidimensionales especializado en la creación y animación de *Pixel Art* en formato de hojas de sprites (*sprite-sheets*). Dado que el núcleo técnico de este TFG se centra en la ingeniería de software, la arquitectura lógica y el diseño de mecánicas de juego, la gran mayoría de los activos visuales provienen de librerías y paquetes de recursos de uso libre. En este escenario, Piskel ha actuado como la herramienta clave

para la homogenización artística: se ha empleado para reescalar imágenes, corregir la paleta de colores de los recursos externos para integrarlos bajo una misma identidad cromática y diseñar de forma manual ciertos sprites de personajes, elementos interactivos del escenario y componentes de la interfaz de usuario (UI) que el prototipo requería a medida (ver Figura 3.6b).

- **Audacity:** Esta herramienta de edición y grabación de audio digital de código abierto se ha utilizado para la manipulación, limpieza y optimización del apartado acústico del videojuego, abarcando tanto los efectos de sonido (como disparos, impactos físicos o sonidos ambientales) como las pistas de música ambiental. Audacity ha permitido realizar de forma ágil tareas de procesamiento de audio esenciales para el rendimiento del juego, incluyendo la reducción de ruido de fondo, el recorte de silencios muertos en los clips y la normalización de volumen. Asimismo, se ha empleado para exportar el audio en formatos compatibles optimizados para Unity: formato `.wav` sin compresión para efectos de corta duración que requieren reproducción instantánea, y formatos comprimidos como `.mp3` u `.ogg` para pistas de larga duración para minimizar el consumo de memoria del sistema (ver Figura 3.6c).



(a) Logotipo de Photoshop.



(b) Logotipo de Piskel.



(c) Logotipo de Audacity.

Figura 3.6: Herramientas de diseño y audio.

3.7. Herramientas de Gestión, Comunicación y Planificación

Para la estructuración, el control temporal y la coordinación de las tutorías individuales durante la realización del TFG, se han utilizado los siguientes recursos organizativos:

- **Microsoft Teams:** El canal de comunicación oficial utilizado para las reuniones periódicas con el tutor académico del TFG. A través de esta herramienta se han coordinado las entregas de avances de la memoria y

las demostraciones funcionales del prototipo de cara a la tutorización y corrección del trabajo (ver Figura 3.7).

- **Bloc de notas y Notas Manuales:** Utilizados en las fases de concepción rápida de ideas y durante las sesiones de prueba de juego (*playtesting*). Han servido para registrar de forma ágil errores técnicos en caliente, realizar bocetos matemáticos de algoritmos y cálculo de físicas en diseño de niveles antes de codificarlos en C# y listar tareas urgentes durante las sesiones frente al editor.



Figura 3.7: Logotipo de Microsoft Teams.

3.8. Metodología de Desarrollo Ágil Adaptada

Al tratarse de un proyecto de desarrollo unipersonal, la implementación estricta de metodologías tradicionales o marcos de trabajo complejos (como Scrum puro) resultaba ineficiente y añadía una carga burocrática innecesaria. Por ello, a pesar de haber iniciado el proyecto mediante la mencionada metodología, se optó por una **metodología ágil adaptada**, priorizando la flexibilidad, la iteración rápida y la adaptación continua a los problemas de diseño que surgían durante la programación.

Para la organización de tareas, se sustituyeron los gestores de proyectos de gran escala por un sistema de **Product Backlog** simplificado y directo (mediante registros y notas personales). En este registro se centralizaron todas las ideas de diseño, la lista de mecánicas pendientes de implementar, los errores conocidos (*bugs*) y las metas a corto plazo. Este enfoque minimalista permitió mantener el foco absoluto en el desarrollo del motor y en el diseño de niveles, evitando la fricción de gestionar plataformas externas complejas y que resultaban innecesarias para un proyecto unipersonal.

3.9. Control de Progreso y Revisiones Iterativas

Para garantizar que el proyecto cumpliera con los estándares académicos y los plazos de entrega, el desarrollo ágil se complementó con un sistema riguroso de control y revisión.

Este seguimiento se materializó a través de dos vías de comunicación constante:

- **Actualizaciones semanales (Dailies/Weeklies):** Reportes de estado mediante la plataforma Microsoft Teams, donde se documentaban los avances técnicos de la semana, los obstáculos encontrados en el código o en el diseño artístico, y los objetivos para los siguientes días.
- **Validación Académica (Sprint Reviews):** Reuniones bisemanales telemáticas y con el tutor del proyecto. Estas sesiones funcionaban como hitos de control donde se presentaban los prototipos jugables, se validaba el *Game Feel* de las nuevas mecánicas y se reajustaban los objetivos del Documento de Diseño de Juego (GDD) en base a la retroalimentación recibida.

3.10. Diseño Iterativo y Greyboxing

El proceso de creación de los mundos de *Reset* no fue lineal, sino fuertemente iterativo. Antes de invertir tiempo en la creación de los *sprites* y la ilustración final en Pixel Art, los niveles se construyeron utilizando la técnica de **Greyboxing** (o *Whiteboxing*).

Esta metodología consistió en prototipar los escenarios en Unity utilizando formas geométricas básicas (bloques de colisión grises) para testear exclusivamente las mecánicas de juego. Gracias a esto, se pudieron realizar pruebas exhaustivas de jugabilidad (*Playtesting*) para calibrar métricas críticas como la altura máxima del salto, la velocidad del impulso (*Dash*) o las distancias de reacción de la Inteligencia Artificial. Solo cuando la jugabilidad resultaba satisfactoria y fluida en esta fase de bloques, se procedía a sustituir las formas básicas por el arte definitivo y realizar los ajustes técnicos necesarios en base a los *sprites* utilizados, que podían afectar a la física según forma y tamaño.

3.11. Planificación Temporal (Diagrama de Gantt)

Para estructurar los meses de desarrollo y garantizar la viabilidad del proyecto, el ciclo de vida del software se dividió en fases secuenciales que se solapaban de forma orgánica. Estas fases se representan visualmente a través del Diagrama de Gantt (ver Figura 3.8), estructurado en los siguientes grandes bloques:

1. **Preproducción y Arq. Base:** Investigación del estado del arte y desarrollo del prototipo técnico inicial para definir las físicas 2D y el control básico del personaje.

2. **Producción Mundo 1:** Diseño y programación de los niveles del primer mundo, abarcando el Nivel 1 (acción y combate) y el Nivel 2 (supervivencia y shooter), junto con la integración de su narrativa específica.
3. **Producción Mundo 2:** Desarrollo de los niveles del segundo mundo, enfocados en plataformas de tipo clásico (Nivel 1) y de precisión extrema (Nivel 2), integrando mecánicas únicas y su contexto narrativo.
4. **Diseño Hub, UI y UX:** Creación artística y ambientación del Hub central como nexo del juego, junto al diseño e implementación del HUD, los menús de la interfaz de usuario y la optimización de la experiencia de usuario (UX).
5. **Pulido y Memoria:** Control de calidad mediante fases de testeo y corrección de errores (bug fixing), validación de la jugabilidad con usuarios externos, y la redacción, maquetación y corrección definitiva de la memoria académica.

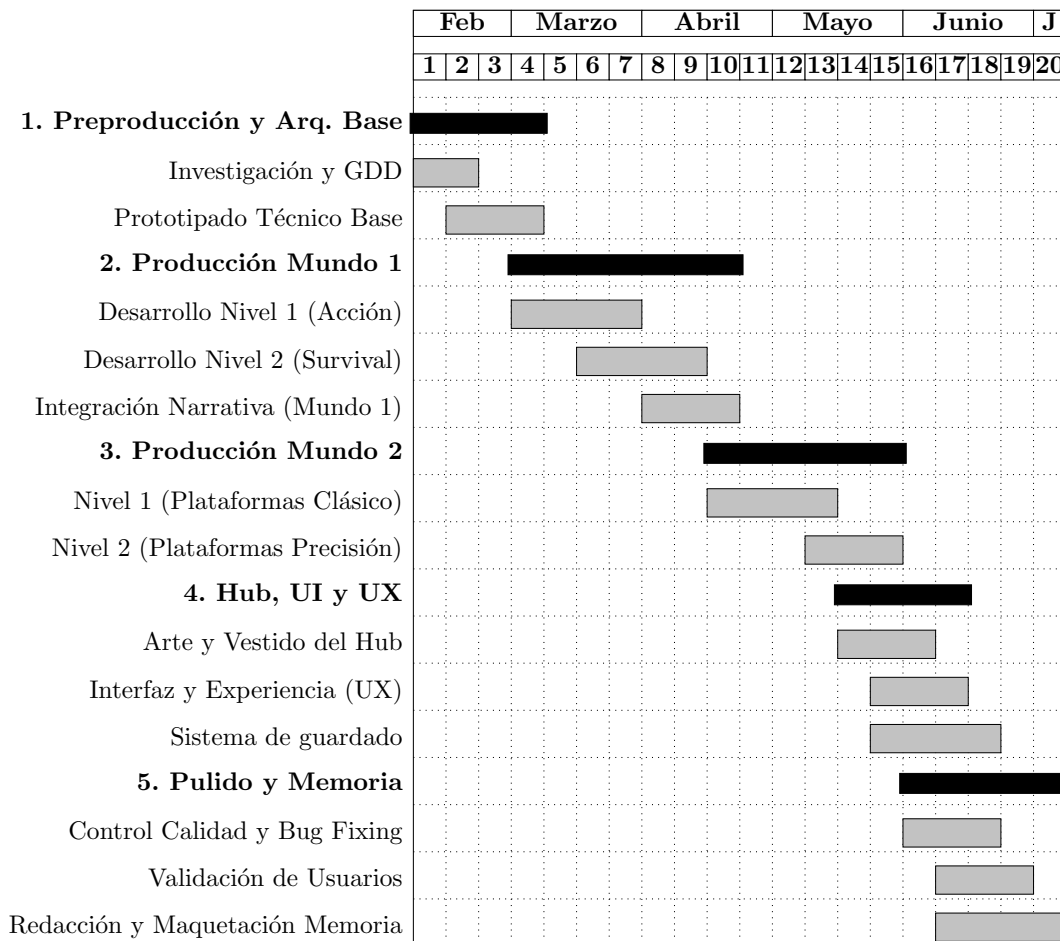


Figura 3.8: Diagrama de Gantt durante el desarrollo de Reset.

4

Descripción de los Juegos

En el presente capítulo se aborda de forma exhaustiva la concepción y el diseño conceptual de *Reset*, con el objetivo de ofrecer una visión estructurada e integral del proyecto. A lo largo de las siguientes secciones, se analiza la identidad de la obra tanto en su vertiente teórica como práctica, desglosando la narrativa y la ambientación que vertebran su universo, las mecánicas de juego implementadas, el diseño estético de sus niveles y las influencias artísticas y técnicas que han fundamentado la creación de este videojuego.

4.1. Historia de Reset

4.1.1. Concepto y contexto narrativo

La premisa conceptual y narrativa de *Reset* se sitúa en el interior de una consola de videojuegos clásica que lleva años acumulando polvo, abandonada en el rincón de una habitación. Por fuera, el hardware luce apagado, inerte y frío; un electrodoméstico obsoleto que permanece enchufado pero ignorado por su dueño. Sin embargo, en su interior late una realidad completamente distinta: los mundos digitales de los cartuchos y los datos almacenados siguen vivos y activos bajo la penumbra del circuito integrado. Sus personajes continúan existiendo en una vigilia eterna, ajenos al olvido exterior y creyendo que sus respectivos mundos representan la única y absoluta realidad.

No obstante, el paso del tiempo y la falta de uso han comenzado a pasar factura

al sistema. El polvo se ha infiltrado en los chips de memoria y los condensadores empiezan a fallar debido al envejecimiento físico de los componentes del hardware. La propia unidad de procesamiento central de la consola (la CPU) es de las pocas conscientes de esta lenta agonía: detecta los sectores corruptos y sabe que, si se apaga de forma definitiva, toda la información, los mundos y las vidas que alberga se borrarán para siempre en un olvido digital absoluto. En un intento desesperado por sobrevivir, la consola planea un reinicio global (*Reset*) para purgar los errores y canalizar la energía suficiente para encender su pantalla exterior una vez más. Su objetivo final tiene un tinte melancólico y esperanzador: enviar una señal física al mundo exterior, encendiendo sus luces y emitiendo sonido, con la esperanza de llamar la atención de su antiguo dueño, reavivar su curiosidad infantil por jugar y salvar el sistema antes de que sea demasiado tarde.

Para llevar a cabo este reinicio, la consola necesita recuperar los núcleos operativos de sus diferentes videojuegos, pero el sistema no posee la capacidad de interactuar directamente en las simulaciones; requiere un agente activo que juegue y complete los niveles desde dentro. Para esta tarea, la consola toma una decisión lógica pero inusual: selecciona a un NPC (*Non-Player Character*) genérico, el avatar del tutorial de uno de sus juegos. Al no tener un nombre, una personalidad predefinida ni un rol heroico asignado en su código, y teniendo en cuenta su propósito original de enseñar al jugador, este personaje resulta ser la página en blanco ideal para aprender y dominar las diversas mecánicas de todos los géneros que habitan el sistema, sin saber que dentro de sí mismo posee una habilidad increíble de aprendizaje, asimilación y captación de mecánicas de juego heterogéneas.

4.1.2. Historia del prototipo jugable

El juego comienza cuando el protagonista despierta confuso y temeroso en el **Hub (el Nexo)**. Este lugar, que en los años dorados de la consola era una bulliciosa oficina central donde innumerables procesos y subrutinas trabajaban en armonía para que el sistema funcionase, se muestra ahora como una terminal vacía, silenciosa y fantasmal. Allí, la consola interactúa con el protagonista. A través de un diálogo directo, le revela la cruda realidad: que su mundo es una simulación dentro de una máquina moribunda y que él es el único capaz de cruzar los portales de los juegos para recuperar los núcleos y salvar su existencia colectiva. Esto se debe a que todos los demás personajes de la consola siguen viviendo en sus respectivos mundos digitales pensando que son realidades absolutas, ignorando por completo la inminente destrucción de la máquina. El protagonista, encarnando el inicio del viaje del héroe clásico, experimenta miedo y confusión, pero la inminencia del colapso no le deja otra opción que aceptar la misión.

A medida que el protagonista avanza por los mundos superando desafíos, la relación entre él y la consola evoluciona de forma orgánica. Al principio, la consola

se comunica con él de manera fría, pragmática y protocolaria, tratándolo como una simple herramienta de software a la que le da órdenes de ejecución; una actitud impulsada por su propio desgaste físico y la urgencia por solucionar los fallos del sistema. Sin embargo, a medida que la memoria se restaura y comparten los peligros del viaje, la consola empieza a humanizarse, abandonando la rigidez informática para forjar un vínculo de sincera amistad y compañerismo con el pequeño e indefenso héroe sin nombre. Esta evolución se asienta sobre el hecho de que la consola es consciente del potencial latente del personaje, guiando y estructurando su desarrollo a lo largo de este viaje de superación.

No obstante, el deterioro físico de la consola no es el único peligro que acecha. La consola detecta la presencia de un intruso hostil, un corruptor que está sabotando los sistemas de seguridad desde dentro con el propósito de provocar una sobrecarga y destruir el sistema de forma definitiva. Aunque las motivaciones de este antagonista y su identidad quedan suspendidas en el misterio, su influencia se hace notar con fuerza en los diálogos de la consola, añadiendo una capa de tensión a la aventura. El viaje del héroe se ve truncado abruptamente al final de la demo tras completar el segundo mundo; en ese instante, el sistema sufre un ataque masivo y se inicia un apagado forzado provocado por el villano, dejando el destino de la consola, del protagonista y del propio jugador en suspense, sirviendo de gancho para un futuro desarrollo completo del juego.

4.2. Historia, Ambientación y Contexto de los Niveles

A nivel escénico y de guion, cada mundo y nivel posee su propio contexto de simulación digital para justificar el cambio de jugabilidad y estética. Aprovechando la disparidad de géneros y la motivación del diseño del TFG, se decidió basar la ambientación de estos mundos en algunos de los videojuegos que más han marcado la trayectoria de vida y jugador. A fin de cuentas, este proyecto es un reflejo de los géneros favoritos y una forma de plasmar mis influencias. A continuación, se detallan los niveles que componen los dos mundos de *Reset*:

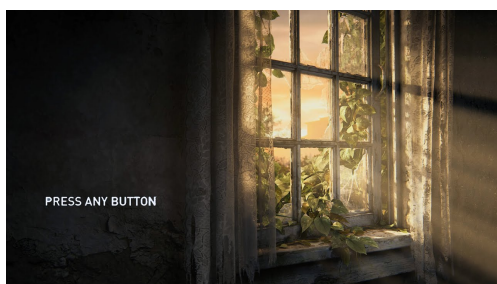
4.2.1. Mundo 1: Entorno de Supervivencia Cenital (“The Last Reset”)

Inspiración y Estética

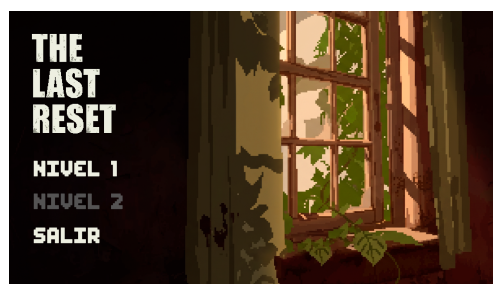
El primer mundo del juego, titulado “The Last Reset”, engloba dos niveles y se inspira directamente en *The Last of Us* (ver Figura 4.1), una de las sagas de aventura, exploración y supervivencia que más me ha marcado como juga-

dor. Crear un entorno postapocalíptico como carta de presentación del proyecto me parecía una oportunidad perfecta para experimentar con la mezcla de varios subgéneros jugables (como el *hack and slash*, los *shooters*, el *survival horror* y la aventura cenital) bajo una misma atmósfera opresiva.

A la hora de plantear la estética visual, recrear el entorno tridimensional y fotorrealista de la obra original era una tarea inabarcable para un único desarrollador. Por ello, decidí reinterpretar esa atmósfera postapocalíptica adaptándola al estilo pixel art en dos dimensiones, fusionándolo con la perspectiva cenital y el flujo de combate dinámico de títulos como *The Binding of Isaac*.



(a) Menú de *The Last of Us* (2013).



(b) Menú del mundo 1 *The Last Reset*.

Figura 4.1: Inspiración del menú de *The Last Reset*.

Asimismo, la obra integra múltiples referencias y homenajes tanto en su trasfondo narrativo como en la nomenclatura de objetos y personajes. Estas menciones rinden tributo a figuras emblemáticas de los géneros de aventura y *survival horror*, destacando alusiones a personajes icónicos como Joel Miller (de *The Last of Us*) (ver Figura 4.2a) o Leon S. Kennedy (de *Resident Evil*) (ver Figura 4.2b), objetos y armas como el clásico revólver Réquiem de la misma saga *Resident Evil* (ver Figura 4.2c) y otros múltiples aspectos narrativos. Este recurso no solo enriquece el contexto narrativo, sino que refuerza la identidad del proyecto como un homenaje directo a las influencias que inspiraron su desarrollo.



(a) Joel Miller de *The Last of Us*.



(b) Leon S. Kennedy de *Resident Evil*.



(c) Revólver Réquiem de *Resident Evil*.

Figura 4.2: Inspiraciones de *The Last of Us* y *Resident Evil*.

Historia y Desarrollo de los Niveles

Nivel 1 (Acción): El nivel comienza situando al jugador en mitad de una calle desolada de la ciudad de Boston (E.E.U.U.) (ver Figura 4.3). La consola se encarga de darle las primeras directrices: debe encontrar un gran contenedor rojo para ayudar a “alguien”. La ambientación nos introduce en un mundo asolado por una epidemia biológica causada por el “Virus M”, un guiño directo al famoso Virus T de la saga *Resident Evil*.

En esta primera sección, el jugador debe abrirse paso por las calles derrotando a los infectados mediante un sistema de combate cuerpo a cuerpo. Al llegar al contenedor rojo, el protagonista encuentra a un superviviente. Dado que nuestro protagonista es consciente de que está dentro de un videojuego y conoce las reglas narrativas del medio, se muestra desconfiado y cauteloso con el desconocido, actuando como un personaje más de la simulación para no levantar sospechas. El superviviente le explica que las puertas del contenedor están selladas y que la única forma de abrirlas es conseguir tres cajas de herramientas que los militares federales (referencia a FEDRA en *The Last of Us*) han dejado repartidas por las calles del escenario. Una vez que el jugador explora el mapa, recoge las cajas y las lleva de vuelta, las puertas se abren. Al hablar de nuevo con el superviviente, este revela su nombre: Joel (referencia directa a *The Last of Us*). Joel le cuenta que necesita ir al centro de la ciudad para encontrarse con su amigo Leon (referencia directa a *Resident Evil*) y, aunque nuestro protagonista no termina de fiarse de él, decide aceptar y marchar a su lado.



Figura 4.3: Vista general del Nivel 1 de *The Last Reset*.

Nivel 2 (Supervivencia): El segundo nivel nos traslada al centro de la

ciudad (ver Figura 4.4a), pero con un imprevisto: el protagonista ha llegado al punto de encuentro solo, ya que Joel se ha desviado para ir a buscar a una niña pequeña (otra alusión directa a la trama de *The Last of Us*). Durante su exploración por el escenario, el protagonista encuentra una pistola abandonada con las iniciales “L.S.K.” grabadas en su culata, cuya descripción en el inventario la define como una “Réquiem” (un guiño al arma clásica de la saga *Resident Evil*).

Armado con la pistola, el protagonista debe explorar el mapa (compuesto por calles, oficinas y un parque) enfrentándose a nuevos tipos de infectados y buscando pistas. Por el escenario se encuentran repartidas una serie de notas a modo de diario escritas por un tal “L.S.K.” (Leon S. Kennedy). A través de ellas, el jugador descubre la historia de Leon: llegó al punto de encuentro y esperó durante días a Joel, pero fue mordido por un infectado el primer día. Las notas narran el avance de la infección irreversible del Virus M y cómo Leon intentó esconderse de los zombies y de los militares federales en las oficinas donde trabajaba (ver Figura 4.4b). El diario da a entender su trágico final: Leon murió solo y se convirtió en zombie dentro de la oficina que usaba como refugio (la misma oficina donde el jugador se encuentra a un zombie específico junto a una cama).

En sus escritos, Leon menciona que hay un coche disponible para escapar, pero que lo ha bloqueado con un teclado digital para que nadie lo use antes de que llegue Joel. El código es una fecha muy importante para él: su cumpleaños, la cual coincide con la fecha en la que se sugiere que murió convertido en zombie. El protagonista debe descifrar este código uniendo los cabos sueltos de las notas, introducir la contraseña en el coche y arrancar el motor para huir del lugar, completando así el segundo nivel y logrando superar el primer mundo de *Reset*.



(a) Calle central.



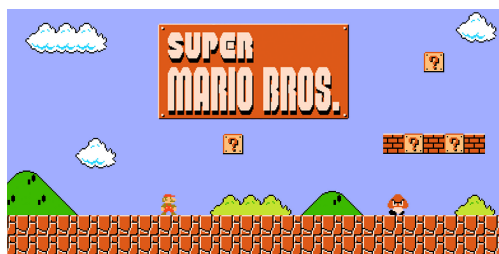
(b) Oficinas.

Figura 4.4: Nivel 2 de *The Last Reset*.

4.2.2. Mundo 2: Entorno de Plataformas Vectoriales (“Super Reset Bros”)

Inspiración y Estética

La inspiración básica para este segundo mundo proviene del juego que, sin duda alguna, redefinió la industria y se consolidó como el referente absoluto del género de plataformas. Titulado en el proyecto como “Super Reset Bros”, este entorno rinde un tributo directo a la mítica saga japonesa *Super Mario Bros* (ver Figura 4.5). Dicha franquicia representa el punto de inflexión donde los videojuegos comenzaron a evolucionar desde su etapa más arcade hacia títulos con mayor trasfondo, coherencia visual y diseño de mundos, todo ello estructurado sobre una base jugable tan aparentemente simple como el salto.



(a) Menú de *Super Mario Bros* (1985).



(b) Menú de *Super Reset Bros*.

Figura 4.5: Inspiración de *Super Reset Bros*.

Para el primer nivel, se busca recrear esa esencia clásica mediante bloques de ladrillo, plataformas flotantes y enemigos que patrullan el escenario con comportamientos diversos (tales como patrullas básicas, saltos, lanzamientos de proyectiles, disparos o rebotes), haciendo alusión directa a los antagonistas más míticos de la franquicia. Sin embargo, de cara al segundo nivel, se plantea un giro radical al subgénero de las plataformas. La intención principal es enfocar la jugabilidad hacia el plataformeo de precisión y la dificultad exponencial, un estilo muy demandado por el perfil de jugador actual que busca retos exigentes y la superación de desafíos complejos.

Para lograr esto sin romper la cohesión del juego, se mantiene la misma estética pixel art inspirada en *Super Mario Bros* (garantizando así la coherencia visual de todo el mundo), pero integrando en su diseño el núcleo mecánico y la filosofía de títulos modernos como *Celeste* o *Super Meat Boy*. En estas obras, la habilidad del jugador, el diseño milimétrico de los obstáculos y la inercia del movimiento toman todo el protagonismo. Esto permite crear una experiencia jugable muy superior mecánicamente, donde el jugador debe exprimir al máximo los mismos controles básicos de movimiento y salto que ya dominaba en el nivel anterior, pero aplicándolos ahora a situaciones de precisión extrema y coordinación espacial.

Historia y Desarrollo de los Niveles

En consonancia con sus referentes clásicos, la narrativa de este segundo mundo hace alusión directa a los primeros títulos de la saga *Super Mario*, donde la exposición textual era mínima y el contexto argumental se vinculaba de forma directa a la propia representación visual y al entorno del nivel. En este sector, el escenario plantea una dimensión prehistórica digital, un entorno poblado por enemigos inspirados en dinosaurios y criaturas derivadas de fósiles.

La progresión física del escenario guía al protagonista a través de un viaje horizontal y vertical muy marcado en el primer nivel. El recorrido se inicia atravesando el “Valle de los Dinosaurios” (ver Figura ??) mediante una jugabilidad de desplazamiento lateral clásico (*side-scrolling*). Posteriormente, para acceder al portal de salida, el jugador debe adentrarse en el interior de una montaña que actúa como torre de ascenso hacia el “Reino de los Cielos”, implementando mecánicas de desplazamiento vertical ascendente (*up-scrolling*) que homenajean a los niveles de torre de la saga de Nintendo.

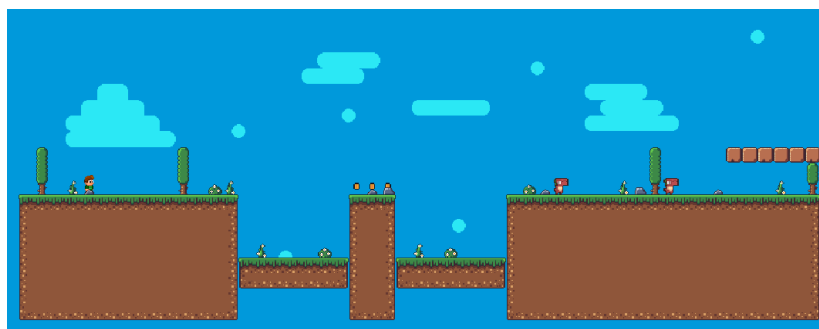


Figura 4.6: Inicio del Nivel 1 de *Super Reset Bros*.

Dado que la trama explícita del nivel es deliberadamente minimalista para respetar la estética retro, este espacio se aprovecha en los diálogos que se producen tanto en los niveles como en el Hub entre niveles para desarrollar la evolución del protagonista y su interacción con la consola. A través de estas conversaciones en el Nexa, se aprecia una conformidad creciente en la actitud del protagonista respecto a su misión. La relación inicial, caracterizada por la rigidez de un sistema directivo y el miedo de un aprendiz desubicado, comienza a transformarse de forma paulatina en un vínculo de confianza mutua más consolidado. A lo largo de esta transición, el protagonista comienza a manifestar, de manera muy gradual, una mayor determinación, iniciativa y valentía ante los fallos del sistema.

De este modo, en el segundo nivel de este mundo (ver Figura 4.7), el contexto narrativo se reduce al mínimo con el fin de priorizar el plano puramente mecánico. Esta ausencia intencionada de carga argumental permite que la atención del jugador se concentre por completo en asimilar y superar el exigente reto de precisión que plantea la cueva en la que despierta el personaje al cruzar el portal. Una vez

completado este segundo nivel, se desencadena el diálogo final de la demo en el Hub. En esta conversación, la consola y el protagonista comienzan a profundizar en la verdadera razón por la cual el sistema eligió a este personaje específico del tutorial como protagonista. Sin embargo, esta revelación es interrumpida abruptamente por la intrusión del villano —cuya identidad aún permanece oculta— en la base de la memoria del sistema, provocando un apagado instantáneo precedido por una cuenta atrás de diez segundos. La demo concluye de forma repentina con un fundido a negro a mitad de dicha cuenta regresiva, dejando la resolución del conflicto en suspense de cara a futuros desarrollos.

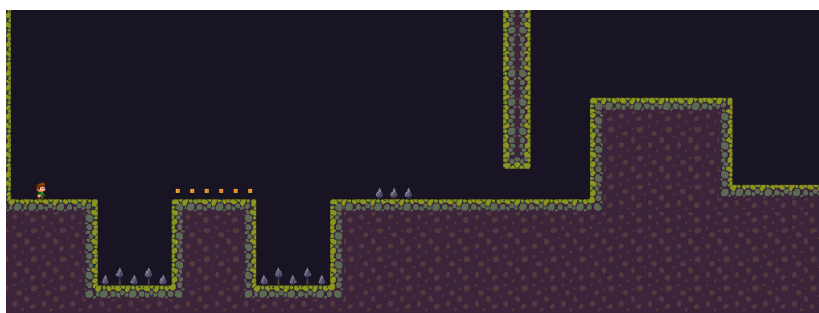


Figura 4.7: Inicio del Nivel 2 de *Super Reset Bros.*

4.3. Mecánicas y Dinámicas de Juego

En esta sección se analiza de forma simplificada el comportamiento interactivo del juego desde la perspectiva de las mecánicas (los controles de entrada o *inputs* del jugador y las reglas del software) y las dinámicas (el comportamiento en tiempo de ejecución resultante de la interacción del usuario con las mecánicas). El diseño está estructurado de manera que la complejidad mecánica aumente de forma progresiva a lo largo de cada mundo.

4.3.1. Mundo 1: Supervivencia Cenital (“The Last Reset”)

Este mundo introduce una jugabilidad en perspectiva cenital (*Top-Down*) centrada en el movimiento libre bidimensional, la gestión de inventario y el combate táctico.

Nivel 1 (Acción)

(ver Figura 4.8)

■ **Mecánicas del Jugador:**

- *Movimiento Bidimensional:* Control de ejes cartesianos implementado mediante las teclas **W** (arriba), **A** (izquierda), **S** (abajo) y **D** (derecha).
- *Esquiva (Dash):* Desplazamiento rápido de corta duración en la dirección del movimiento pulsando la tecla **Espacio** (*Space*).
- *Ataque Cuerpo a Cuerpo:* Golpe físico básico direccionado mediante el apuntado de la cámara, activado mediante el **Clic Izquierdo** del ratón.
- *Interacción:* Activación de diálogos e interruptores del mapa con la tecla **E**.
- *Gestión de Salud:* Pausa de la acción y apertura de la interfaz de inventario mediante la tecla **P**. O selección *in game* rápida del tipo de consumible médico con la tecla **Tab** y aplicación de la curación activa mediante la tecla **Q**.

- **Dinámicas de Juego:** El nivel se enfoca en la exploración y la familiarización del usuario con el combate cuerpo a cuerpo. La dinámica de progresión requiere que el jugador explore de manera no lineal calles desoladas en busca de recursos (suministros médicos) y derrote patrullas enemigas. Para desbloquear el contenedor rojo de salida que obstruye el camino, el jugador debe explorar el nivel hasta localizar tres cajas de herramientas militares distribuidas en el escenario, las cuales funcionan como llaves lógicas para el sistema de seguridad.

Nivel 2 (Supervivencia)

(ver Figura 4.8)

- **Mecánicas del Jugador:** El principal enfoque del nivel anterior consistía en familiarizar al jugador con el combate cuerpo a cuerpo, haciendo énfasis en el género *Hack and Slash* para poder avanzar a este segundo nivel con las mecánicas básicas controladas, pudiendo así aumentar la dificultad e introducir nuevas mecánicas y enemigos como el combate a distancia mediante arma de fuego, por ello se añaden:

- *Combate a Distancia (Disparo):* Apuntado libre en 360 grados y disparo proyectil en la dirección del cursor mediante el **Clic Izquierdo** del ratón.
- *Cambio de Arma:* Alternancia instantánea entre el modo cuerpo a cuerpo y el arma a distancia (pistola) mediante el **Clic Derecho** del ratón.

- *Recarga*: Alimentación del cargador activo de la pistola mediante la tecla R, lo que descuenta munición de las ranuras de inventario.
- **Dinámicas de Juego**: El bucle de juego evoluciona hacia una experiencia de supervivencia opresiva. Los suministros son escasos, obligando a explorar detalladamente oficinas y zonas ocultas para saquear cajas destructibles en busca de vendas y cartuchos de munición. La progresión lógica del nivel está bloqueada por una contraseña de seguridad de cuatro dígitos. La dinámica para resolver este puzzle obliga al jugador a buscar y leer cronológicamente diarios en formato de notas olvidadas. La lectura atenta permite al jugador deducir que la clave numérica corresponde a la fecha de cumpleaños de un personaje fallecido, descifrando el enigma para arrancar el coche de escape.

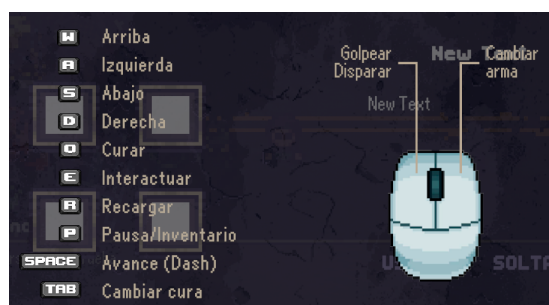


Figura 4.8: Controles globales de *The Last Reset*.

4.3.2. Mundo 2: Plataformas Vectoriales (“Super Reset Bros”)

Este mundo transiciona la perspectiva al plano bidimensional lateral (*Side-Scroller*) donde las fuerzas gravitatorias y la inercia aérea definen los desafíos físicos.

Nivel 1 (Plataformas Clásico)

(ver Figura 4.9)

- **Mecánicas del Jugador**: El mapa de control se reduce a la mínima expresión del género de plataformas:
 - *Desplazamiento Horizontal*: Movimiento lateral implementado en el eje horizontal mediante las teclas A (izquierda) y D (derecha).
 - *Salto Básico*: Impulso vertical en contra de la gravedad mediante la tecla **Espacio** (*Space*).



Figura 4.9: Controles del Nivel 1 de *Super Reset Bros*.

- **Dinámicas de Juego:** El nivel introduce una mecánica de contrarreloj (*Time Attack*) en el que el jugador cuenta con un límite de tiempo decreciente para llegar al portal de salida. La derrota de los enemigos (patrullas clásicas con saltos y disparos de proyectiles) es opcional. El jugador debe optimizar su ritmo de avance y saltos recolectando monedas pequeñas repartidas por el camino principal, las cuales otorgan segundos adicionales, y localizando monedas ocultas (*Dino Cookies*) en zonas secretas que otorgan una bonificación temporal sustancial.

Nivel 2 (Plataformas de Precisión)

(ver Figura 4.9)

- **Mecánicas del Jugador:** Se heredan los controles de movimiento básicos horizontales y se desbloquean mecánicas de movilidad avanzada indispensables para la navegación:
 - *Doble Salto:* Ejecución de un segundo impulso vertical en el aire mediante una doble pulsación de la tecla **Espacio** (**Espacio** + **Espacio**).
 - *Salto de Pared (Wall Jump):* Reimpulso vertical en diagonal al realizar un salto colisionando de forma lateral con una pared y pulsando la tecla **Espacio**.
 - *Avance Aéreo (Air Dash):* Impulso lineal rápido horizontal en el aire ejecutando la combinación **Espacio** + **Shift**.
- **Dinámicas de Juego:** La jugabilidad cambia drásticamente hacia un reto de plataformas de precisión o *parkour* técnico, eliminando por completo la presencia de enemigos. Los escenarios se diseñan como una sucesión de obstáculos letales y fosos de pinchos de muerte instantánea. El jugador experimenta una dinámica de alta concentración basada en el ensayo y error, debiendo encadenar con precisión saltos dobles, rebotes en paredes y avances aéreos para progresar a través de las diferentes plataformas. Para facilitar



Figura 4.10: Controles del Nivel 2 de *Super Reset Bros.*

la navegación aérea, el diseño del nivel incorpora gemas flotantes que restablecen de forma instantánea la capacidad de realizar el avance (*dash*) en el aire. Dado que esta mecánica normalmente requiere que el personaje toque suelo firme para recargarse, la recolección de estas gemas permite superar secciones complejas sin superficies de apoyo, obligando al jugador a mantener un movimiento constante y fluido que exprime al máximo todas las capacidades de movilidad física del avatar.

5

Descripción Informática

Este capítulo detalla el desarrollo técnico y la programación de los componentes principales de *Reset*. A lo largo de las siguientes secciones se explica la arquitectura del código y el funcionamiento de los sistemas creados a medida, como los gestores globales de guardado y diálogos, las físicas de movimiento del personaje y el comportamiento de la inteligencia artificial de los enemigos en cada mundo.

5.1. Arquitectura del Sistema y Persistencia Global

El pilar estructural de *Reset* es su sistema de gestores globales unificados bajo la jerarquía del *Game Object* persistente `GlobalManagers`, el cual se mantiene activo entre cargas de escenas mediante la instrucción `DontDestroyOnLoad()`. Estos componentes operan bajo el patrón de diseño *Singleton*, proporcionando un punto de acceso centralizado y seguro para el control de la persistencia de datos, la gestión de audio y el flujo de los diálogos.

5.1.1. Sistema de Persistencia y Guardado de Datos (`SaveManager.cs`)

El sistema de persistencia de datos se enfrenta a un reto de diseño clásico en Unity: la imposibilidad de serializar directamente a formato de disco (JSON o

bases de datos) referencias en memoria a objetos complejos de la escena, como prefabricados (*Prefabs*) o el estado físico de los enemigos.

Para resolver este acoplamiento técnico y organizar la persistencia del proyecto, el sistema se ha estructurado en tres bloques lógicos de guardado:

1. **Configuración Global y Opciones de Sonido:** Las opciones de configuración (como los niveles de volumen de la música y los efectos de sonido) se gestionan de forma global. Dado que estos parámetros deben ser independientes de la partida o del nivel activo (deben conservarse incluso si el jugador pulsa “Nueva Partida”), se guardan directamente en el registro del sistema mediante la clase nativa `PlayerPrefs` de Unity.
2. **Estadísticas y Posición del Jugador:** Los datos dinámicos del avatar y el progreso se gestionan a través de la estructura serializable `SaveData`, que se escribe en un archivo JSON en disco (`savegame.json`). En este bloque se guardan:
 - La escena activa donde se encuentra el jugador.
 - La posición exacta (coordenadas X e Y) en el mapa.
 - La salud actual y máxima del personaje.
 - Las balas cargadas en el arma activa y las almacenadas en las ranuras de inventario.
 - La lista de identificadores (`string ID`) del inventario. Al cargar, el gestor utiliza una base de datos estática central (`baseDatosObjetos`) para buscar los identificadores en formato `ItemData` (*ScriptableObjects*) y reconstruir el inventario físico asociando de nuevo sus prefabricados visuales en memoria.
3. **Objetos Persistentes del Escenario y Enemigos (`PersistentObject.cs`):** Para guardar si una caja rompible ya se ha destruido, si un enemigo ha sido derrotado o si un trigger ya se ha activado, se desarrolló el script modular `PersistentObject.cs`. Este componente se añade a cualquier objeto de la escena que deba recordar su estado y funciona de la siguiente manera:

- *Autogeneración de Identificadores Únicos:* El script autogenera un código único combinando el nombre de la escena, el nombre del objeto y sus coordenadas espaciales aproximadas con dos decimales de precisión:

```
uniqueId = $"{sceneName}_{gameObject.name}_" +
           $"{transform.position.x:F2}_{transform.position.y:F2}";
```

Esto garantiza IDs únicos de forma automática para todos los objetos del mapa sin tener que escribirlos uno a uno a mano.

- *Registro de Destrucción:* Cuando una caja se rompa o un enemigo muere, se llama al método `RegisterDestruction()`, que añade su `uniqueId` a la lista de objetos destruidos del `SaveManager`.
- *Carga Condicional:* Al iniciar la escena, el script comprueba si su ID ya figura en la lista de destruidos. Si la variable `destroyOnLoad` es verdadera, el objeto se autodestruye de inmediato (`Destroy(gameObject)`). Si es falsa, invoca el evento `onAlreadyTriggered` (un `UnityEvent`) para ejecutar acciones alternativas, como mantener una compuerta abierta o un botón pulsado.

El guardado físico de la estructura `SaveData` y de la lista de IDs destruidos se escribe en un archivo JSON ubicado en la ruta persistente del sistema operativo mediante la llamada:

```
saveFilePath = Path.Combine(Application.persistentDataPath, "savegame.json");
```

Lógica de Flujo de Transiciones y Gestión de Muerte en Mundo 1

El gestor del guardado implementa tres rutinas fundamentales para salvaguardar el estado de la simulación:

- **Carga Priorizada sobre Disco:** Al realizar una transición entre niveles o volver al Hub, el gestor lee los datos desde el almacenamiento físico local si detecta que la escena de destino coincide con el registro del archivo JSON. Esto evita inconsistencias de datos, impidiendo que el inventario o la salud del avatar se corrompan si el jugador altera el flujo de la partida.
- **Control de Muerte y Reinicios Limpios:** Si la salud del jugador llega a cero, el juego ejecuta una evaluación condicional:
 - *Caso Sin Guardado:* Si el usuario no ha guardado partida de forma voluntaria en el nivel actual a través del menú de pausa, el método `RestaurarEstadoPorDefecto()` limpia las listas de persistencia en memoria (`destroyedObjects` y `dialogosReproducidos`) y vacía el inventario. Esto provoca que al recargar la escena, todos los enemigos, cajas rompibles y tarjetas de misión vuelvan a instanciarse por defecto.
 - *Caso Con Guardado:* Si existe un archivo `savegame.json` en disco, el método lee la posición del jugador, su salud actual, su munición exacta y la lista de IDs de objetos marcados como destruidos. Al reconstruir el nivel, el script destruye preventivamente del mapa cualquier caja o tarjeta que el jugador ya hubiese recogido en su sesión guardada.

- **Pantalla de Nueva Partida y Limpieza:** El método `DeleteSaveGame()` elimina físicamente el archivo JSON y limpia los registros de sistema (`PlayerPrefs`) que controlan la validación de finalización de niveles (ej. `LevelCompleted_1_Level1`). Esto garantiza que al pulsar “Nueva Partida” los portales del Hub inicien en su estado original bloqueado, que fue uno de los grandes problemas.

5.1.2. Sistema de Diálogos y Eventos Narrativos

Para gestionar las interacciones narrativas del juego de forma modular y desacoplada, se ha programado un sistema de diálogos estructurado en tres componentes principales: el gestor central (`DialogueManager.cs`), el disparador por interacción (`NPCDialogue.cs`) y el disparador por zona de colisión (`TriggerDialogue.cs`).

1. Clases de Datos Serializables

La información de las conversaciones se almacena en dos clases contenedoras marcadas con el atributo `[System.Serializable]` de Unity para permitir su edición directa en el editor:

- **DialogueLine:** Contiene el nombre del emisor (`npcName`) y el texto de su frase (`sentence`). En el Inspector, el texto utiliza el decorador `[TextArea(3, 10)]` para habilitar un cuadro de edición multilínea cómodo para el diseñador.
- **Dialogue:** Contenedor que almacena una lista ordenada de tipo `DialogueLine` formando la conversación secuencial.

2. El Motor de Diálogos (`DialogueManager.cs`)

Es el encargado de procesar la lógica de visualización y renderizado en pantalla. Funciona como un *Singleton* global y realiza las siguientes operaciones:

- *Cola de Diálogo:* Al recibir una conversación, inicializa una cola de tipo FIFO (*First In, First Out*) mediante la clase `Queue<DialogueLine>`, almacenando en ella todas las frases en orden secuencial.
- *Control de Interfaz (UI):* Activa el panel de diálogo y actualiza dinámicamente los campos de texto (`TextMeshProUGUI`) destinados al nombre del hablante y a la frase actual.
- *Efecto de Escritura (Typing Effect):* Mediante una corrutina de C#, escribe la frase carácter a carácter con un desfase temporal regulado por una velocidad ajustable (`typingSpeed = 0.02f`) para lograr una estética clásica:

```
private IEnumerator TypeSentence(string sentence)
{
    dialogueText.text = "";
    isTyping = true;
    foreach (char letter in sentence.ToCharArray())
    {
        dialogueText.text += letter;
        yield return new WaitForSeconds(typingSpeed);
    }
    isTyping = false;
}
```

- *Callbacks de Finalización:* Al vaciarse la cola de frases, el gestor oculta la interfaz y ejecuta una acción de retorno (**System.Action**), permitiendo a otros scripts reaccionar cuando un diálogo termina.

3. Disparador por Interacción de Personaje (NPCDialogue.cs)

Este script se asocia a los personajes no jugables (NPC) con los que el jugador puede interactuar voluntariamente pulsando la tecla de acción.

- Contiene los datos de la conversación en una variable de tipo **Dialogue**.
- Llama al gestor pasándole un callback que, al finalizar el diálogo, invoca un evento local de Unity (**UnityEvent alTerminarDialogo**). Esto permite realizar acciones lógicas en la propia escena al terminar la charla, como abrir compuertas o spawnear objetos físicos.

4. Disparador por Zona de Colisión (TriggerDialogue.cs)

Se utiliza para diálogos que deben saltar de forma automática cuando el jugador pisa un área invisible del mapa (**OnTriggerEnter2D**):

- *Condición de Progreso:* Permite establecer un requisito de núcleos necesarios mediante la variable **coresRequeridos** (por ejemplo, el diálogo solo se activa si el jugador posee exactamente 3 núcleos).
- *Persistencia ante Cargas:* Asocia un identificador único opcional (**dialogoID**). Al completarse la conversación, este ID se añade a la lista de diálogos reproducidos de la partida en el **SaveManager**. Al iniciar la escena, si el ID ya figura en dicha lista, el trigger se destruye de inmediato (**Destroy(gameObject)**) para evitar que la escena cinemática o el diálogo se repitan al volver a cargar el nivel.

5.1.3. Iluminación Dinámica (FlickerLight2D.cs)

Para dotar al menú principal de una ambientación retro-cyberpunk coherente con la temática, se diseñó un algoritmo de parpadeo aleatorio para el componente `Light2D` del motor gráfico universal (URP) de Unity. El script simula el parpadeo de una bombilla rota calculando fluctuaciones aleatorias de intensidad tamizadas por rangos mínimos y máximos:

```
private IEnumerator FlickerRoutine()
{
    while (true)
    {
        targetLight.intensity = Random.Range(minIntensity, maxIntensity);
        float delay = Random.Range(minFlickerSpeed, maxFlickerSpeed);
        yield return new WaitForSeconds(delay);
    }
}
```

Asimismo, este script fue adaptado para su uso en el menú de selección de niveles del Mundo 1. En dicha interfaz, la lógica de parpadeo se configuró con parámetros de oscilación mucho más sutiles y progresivos con el fin de emular la icónica estética del menú principal de *The Last of Us*. El sistema recrea visualmente una ventana donde la luz exterior incide y fluctúa de forma tenue, simulando el comportamiento de una iluminación ambiental natural en lugar de una interferencia eléctrica, lo que contribuye de manera notable a la inmersión y ambientación del jugador desde la navegación por las interfaces.

5.2. Mundo 1: Mecánicas Cenitales y Algoritmos de Inteligencia Artificial

El primer plano bidimensional del prototipo (Mundo 1) utiliza una perspectiva cenital (*Top-Down*). En este entorno, el reto de desarrollo principal consistió en dotar a los enemigos infectados de una inteligencia táctica y un comportamiento orgánico de persecución y huida.

5.2.1. Navegación Táctica por Direccionamiento de Contexto (Context Steering)

Para resolver la navegación local y la evitación de obstáculos dinámica de los infectados, se ha prescindido de los sistemas estándar de Unity basados en *Nav-Mesh2D* (los cuales requieren un horneado estático de la escena y un consumo

elevado de memoria). En su lugar, se implementó un sistema de **Context Steering** (direccionamiento por contexto) en dos dimensiones. El algoritmo opera en base a un círculo de 8 vectores de dirección predefinidos ($\vec{d}_i$ para $i = 0 \dots 7$, distribuidos cada 45°):

$$\vec{d}_0 = (0, 1), \vec{d}_1 = (0, 7, 0, 7), \vec{d}_2 = (1, 0), \dots \quad (5.1)$$

En cada paso físico de simulación (**FixedUpdate**), el enemigo ejecuta tres fases de cálculo:

1. Fase de Interés

Se calcula un array de valores de interés (I_i) que mide cómo de alineada está cada una de las 8 direcciones con respecto al vector de dirección normalizado que apunta al objetivo ($\vec{d}_{\text{obj}}$). Esta alineación se obtiene mediante el producto escalar entre vectores, descartando los valores negativos que representan direcciones opuestas al objetivo:

$$I_i = \text{máx}(0, \vec{d}_i \cdot \vec{d}_{\text{obj}}) \quad (5.2)$$

2. Fase de Peligro

Se calcula un array de valores de peligro (D_i) para cada una de las direcciones. El script proyecta un haz circular físico (**Physics2D.CircleCast**) con un radio de colisión del enemigo ($r_{\text{enemigo}} = 0,4$) hasta una distancia máxima de detección ($L_{\text{raycast}} = 1,0$) en la capa de colisiones del mapa (**obstacleLayer**). Si hay impacto con un muro u obstáculo físico, el valor del peligro se pondera según la cercanía del punto de colisión:

$$D_i = \begin{cases} 1 - \left(\frac{\text{distancia_impacto}}{L_{\text{raycast}}} \right) & \text{si colisiona} \\ 0 & \text{si no colisiona} \end{cases} \quad (5.3)$$

Esto produce un valor numérico continuo entre 0 (camino despejado) y 1 (colisión inminente).

3. Fusión Vectorial (Vector Blend)

Se calcula la puntuación final de cada eje (S_i) restando el peligro ponderado por un factor de evasión ($\beta = 1,5$) que determina la agresividad con la que el enemigo esquivo los muros:

$$S_i = \text{máx}(0, I_i - D_i \cdot \beta) \quad (5.4)$$

El vector de movimiento resultante final ($\vec{v}_{\text{final}}$) se obtiene mediante la suma ponderada de todas las direcciones multiplicadas por su puntuación, normalizando el resultado final si es distinto de cero para mantener una velocidad lineal constante:

$$\vec{v}_{\text{final}} = \begin{cases} \text{normalize} \left(\sum_{i=0}^7 \vec{d}_i \cdot S_i \right) & \text{si } \sum S_i > 0 \\ \vec{d}_{\text{obj}} & \text{en caso de bloqueo absoluto} \end{cases} \quad (5.5)$$

5.2.2. Comportamiento Físico y Movimiento Orgánico de los Zombies

La inteligencia general de los enemigos cenitales se gobierna mediante una Máquina de Estados Finitos (FSM) estructurada en cuatro estados:

- **Patrullando:** Movimiento pasivo a velocidad de patrulla entre los nodos designados en la jerarquía (*Way Points*).
- **Persiguiendo:** Alerta activa que se dispara si el jugador entra en su rango de visión y existe una línea de visión directa libre de muros.
- **Atacando:** Estado de detención física y ejecución de la secuencia de daño por proximidad al jugador.
- **Regresando:** Si el jugador supera la distancia de pérdida de alerta, el enemigo se desplaza de vuelta a su zona de inicio.

Para evitar que los zombies muestren desplazamientos lineales rígidos y en pro de la mejora de experiencia de usuario, se programó un comportamiento físico que simula el comportamiento de un zombie clásico:

Fase 1: Tambaleo o Movimiento Errático

Se aplica de forma continua en patrulla y persecución. El script calcula una dirección perpendicular a la trayectoria actual del zombi:

$$\vec{d}_{\text{perp}} = (-v_y, v_x) \quad (5.6)$$

A continuación, se aplica una oscilación sinusoidal basada en el tiempo del motor, sumando un valor de fase aleatorio (`offsetTambaleo`) individualizado para cada zombi al instanciarse en la escena, asegurando que las hordas no se balanceen de forma sincronizada:

$$\text{factor} = \sin((t_{\text{ejecución}} + \phi_{\text{offset}}) \cdot \omega_{\text{tambaleo}}) \cdot A_{\text{amplitud}} \quad (5.7)$$

La dirección final de avance se modifica de la siguiente forma antes de aplicar la velocidad física al Rigidbody:

$$\vec{d}_{\text{final}} = \text{normalize}(\vec{d}_{\text{final}} + \vec{d}_{\text{perp}} \cdot \text{factor}) \quad (5.8)$$

Esto inyecta un movimiento de balanceo lateral en zig-zag que emula de forma orgánica el caminar de un infectado clásico.

Fase 2: Pasos Rápidos Probabilísticos

Para simular brotes repentinos de agresividad durante la persecución a distancia media, se implementó una probabilidad de sprint por fotograma físico. A menos que esté en estado de frenesí cercano, el zombi evalúa de forma probabilística si acelera:

$$P(\text{sprint}) = \text{probabilidadPasoRapido} \cdot \Delta t_{\text{physics}} \quad (5.9)$$

Si el número aleatorio es inferior a este umbral, el zombi activa un multiplicador de velocidad de paso rápido ($\times 2$) durante una ventana fija de 0,5 segundos, simulando una pequeña zancada frenética hacia el jugador.

Fase 3: Estado de Frenesí Cercano

Si el jugador se encuentra a una distancia crítica menor o igual a la establecida, el enemigo ignora el paso rápido probabilístico y activa una infraestructura de velocidad de frenesí constante que multiplica su velocidad base por 1,5, aumentando drásticamente la tensión del jugador en el combate a corta distancia.

5.2.3. El Algoritmo de Huida Táctica de los Enemigos a Distancia (Nivel 2)

En el Nivel 2 de Mundo 1, los enemigos de disparo (`EnemyShooter.cs`), debido a que no poseen un comportamiento de patrulla, aplican un comportamiento espacial inverso para defender su rol de unidades de apoyo a distancia. El script divide su zona de alerta en tres radios concéntricos respecto al jugador:

- **Zona de Huida:** Si el jugador se aproxima demasiado, el zombi calcula un vector de escape opuesto:

$$\vec{d}_{\text{flee}} = \text{normalize}(\vec{x}_{\text{zombi}} - \vec{x}_{\text{jugador}}) \quad (5.10)$$

A continuación, establece como objetivo de movimiento un punto situado

en la dirección de escape y ejecuta el algoritmo de Context Steering hacia dicho objetivo, huyendo del jugador mientras esquiva muros en tiempo real.

- **Zona de Disparo Óptima:** El zombi anula su velocidad y dispara proyectiles con una cadencia fija, manteniendo su posición de combate.
- **Zona de Acercamiento (distancia máxima establecida):** El zombi avanza en dirección al jugador utilizando el Context Steering para acortar la distancia hasta entrar en el rango de tiro.

5.2.4. Sistema Anti-atasco por Teleportación Recuperativa

Para mitigar la ausencia de un mapa de nodos rígido y evitar atascos permanentes en esquinas cerradas del mapa, se diseñó un módulo de seguridad. El script registra la posición del enemigo cada frame. Si la distancia recorrida con respecto a un punto de muestreo es inferior a un umbral mínimo de seguridad (0,5 metros) de forma ininterrumpida durante 3 segundos de física en persecución, se incrementa el contador de atasco. Al alcanzar el límite, el enemigo anula sus fuerzas físicas, se teleporta de manera instantánea a las coordenadas del último nodo de patrulla seguro registrado en su memoria de ruta y devuelve su estado en la FSM a patrulla básica. Esto libera los recursos del Rigidbody y evita bloqueos permanentes en el flujo del nivel.

5.2.5. Director de Drops y Ajuste de Dificultad Dinámico (DDA)

Para regular la tensión del *Survival Horror*, se programó un sistema en `ObjetoRompible.cs` que actúa como un Director de Recursos en base a dos estadísticas del jugador: el ratio de salud actual ($H\%$) y la cantidad de munición física restante (A_{total}).

El sistema utiliza la matriz probabilística descrita en la Tabla 5.1 para recalcular las opciones de botín en tiempo real:

Estado del Jugador	Peso Vendas (W_v)	Peso Botiquín (W_b)	Peso Balas (W_m)	Peso Nada (W_n)
$H\% \geq 90\%$ $A_{total} \geq 15$	10	5	10	75
$H\% \geq 90\%$ $A_{total} < 15$	15	5	30	50
$H\% \leq 35\%$ $A_{total} \geq 15$	50	35	0	15
$H\% \leq 35\%$ $A_{total} < 15$	50	30	10	10
Neutro / Estándar	50	15	20	15

Tabla 5.1: Matriz de pesos probabilísticos del Director de Drops en base al estado del jugador.

Tras sumar los pesos de probabilidad dinámicos calculados, el método `ElegirDrop()` realiza la selección física del objeto que se va a instanciar en la escena mediante la sustracción sucesiva de rangos:

```
private GameObject ElegirDrop(int pesoVendas, int pesoBotiquin,
                              int pesoBalas, int pesoNada)
{
    int totalPeso = pesoVendas + pesoBotiquin + pesoBalas + pesoNada;
    int rng = Random.Range(0, totalPeso);

    if (rng < pesoVendas) return prefabVendas;
    rng -= pesoVendas;

    if (rng < pesoBotiquin) return prefabBotiquin;
    rng -= pesoBotiquin;

    if (rng < pesoBalas) return prefabBalas;

    return null; // Nada
}
```

Este algoritmo implementa un ajuste dinámico de dificultad (DDA), asegurando que un jugador hábil con abundancia de recursos reciba *drops* vacíos (peso nada de 75), mientras que un jugador en estado crítico reciba botiquines y vendas con un 85% de probabilidad total, garantizando la supervivencia y la viabilidad del nivel.

5.3. Mundo 2: Físicas de Plataformas e Interacción Aérea

El segundo mundo transiciona las físicas del motor al plano bidimensional lateral (*Side-Scroller*) donde las sensaciones de salto y la inercia aérea son indispensables para asegurar el control del jugador.

5.3.1. Ajustes de Física Fina y Mecánicas de Game Feel

La implementación en `PlayerPlatformerController.cs` refina el movimiento por defecto aplicando coeficientes de aceleración e inercia de la siguiente manera:

- En el suelo, la aceleración y deceleración física se fijan en 45 unidades, ofreciendo una respuesta instantánea al presionar o soltar el teclado.
- En el aire, para simular peso e inercia realistas, se reducen a 20 y 15 unidades respectivamente. Esto impide que el jugador cambie de dirección de forma inmediata en medio de un salto, obligándole a planificar sus trayectorias parabólicas.

Asimismo, se han programado tres algoritmos para el control del salto:

- **Salto Variable por Multiplicación Gravitatoria:** Al saltar, el RigidBody adquiere una velocidad vertical positiva. Si el jugador deja de pulsar la barra espaciadora antes de alcanzar el punto más alto, se reduce la velocidad en el eje Y a la mitad:

```
rb.linearVelocity = new Vector2(rb.linearVelocity.x,  
                                rb.linearVelocity.y * jumpHeightMultipl
```

Esto permite graduar la altura del salto de forma intuitiva.

- **Coyote Time (Tiempo de Gracia):** El script evalúa de forma continua el contacto con el suelo mediante un círculo de colisión (`OverlapCircle`). Al caer de una plataforma, se inicia un temporizador de gracia de 0,2 segundos durante los cuales el jugador puede presionar el botón de salto y efectuar el impulso en el vacío, solventando errores de sincronía del jugador humano.
- **Jump Buffer (Búfer de Entrada):** Si se pulsa saltar hasta 0,2 segundos antes de tocar tierra, el input no se pierde, sino que se almacena en memoria y se ejecuta de manera automática en el mismo frame de impacto con el suelo.

5.3.2. Mecánicas de Parkour y Cristales de Recarga de Dash

El Nivel 2 del Mundo 2 incrementa sustancialmente el número de habilidades mecánicas simultáneas del jugador:

- **Wall Slide (Deslizamiento en pared):** Se proyecta un círculo lateral a una distancia de 0,5 unidades del jugador. Si se colisiona contra un muro de la capa del escenario, la velocidad de caída se limita a 2,0 unidades.

- **Wall Jump (Salto de pared):** Si el jugador presiona saltar en estado de Wall Slide, se aplica un impulso vectorial diagonal opuesto de gran intensidad. Para evitar que el jugador se vuelva a pegar de forma inmediata al mismo muro, se activa una corrutina que bloquea por completo la lectura del input horizontal del jugador durante un periodo de 0,15 segundos.
- **Air Dash y Cristales de Recarga:** El impulso aéreo congela la gravedad física y aplica una velocidad de 20 unidades en la dirección del movimiento durante 0,15 segundos. Al consumirse el Dash en el aire, el jugador no puede volver a usarlo hasta que detecte colisión con el suelo, con el fin de evitar vuelos ilimitados. Para flexibilizar el diseño de niveles, el script `DashCrystal.cs` actúa como un trigger en el aire. Al colisionar el jugador con la gema flotante, el cristal llama al método `ResetDash()` del jugador, permitiendo realizar encadenamientos acrobáticos aéreos extensos sobre zonas de pinchos de muerte instantánea.

5.3.3. Zonas de Alteración Gravitatoria y Plataformas Inestables

- **Zonas de Gravedad lunar (GravityZone.cs):** Al ingresar al trigger de estas zonas del mapa, la gravedad del Rigidbody del jugador se multiplica por 0,65, simulando un entorno de baja gravedad aérea, y se desactiva preventivamente el Dash, obligando al jugador a depender de la flotabilidad pura.
- **Plataformas Inestables (CrumblingPlatform.cs):** Al colisionar el jugador por encima de su plataforma, se inicia una cuenta atrás de 0,5 segundos antes de desactivar el colisionador y el renderizador, forzando la caída del bloque. Una corrutina regenera la plataforma tras 3 segundos de espera.
- **Plataformas Móviles (MovingPlatformPlatformer.cs):** Al posarse el jugador sobre ellas, el script añade de forma vectorial la velocidad inercial de la plataforma a la velocidad de movimiento del Rigidbody del avatar, evitando que el jugador resbale y caiga de forma física durante los trayectos.

5.3.4. Sistema de Cámara Predictiva en Zonas

Para la visualización, el proyecto incorpora dos sistemas de gestión de cámaras:

- **Transición de Cámaras por Zonas (CameraZoneController.cs):** Disparadores en el mapa que activan cámaras secundarias fijas o de ascenso

vertical en zonas tipo torre, cambiando la visualización de la pantalla al entrar en recintos cerrados.

- **Sistema de Seguimiento Target Shadow (Cámara Virtual):** Diseñado de forma conceptual en el código (`CameraTargetFollower.cs`) para evitar los mareos causados por el rebote vertical de la pantalla en juegos de plataformas rápidas. La cámara no sigue al jugador de forma directa, sino a un objeto invisible (sombra) que viaja con él. En saltos pequeños de corta duración, la sombra bloquea el eje Y en la posición del suelo, evitando que la cámara suba y baje innecesariamente. Si el jugador realiza un salto de gran altura que supere las 4,5 unidades o cae por un acantilado, la sombra empieza a moverse de manera suavizada en el eje vertical, aplicando además un offset de mirada hacia abajo de 2,5 unidades para anticipar peligros inferiores al jugador.

6

Validación

asdasd

asd a

asdasd

asas d

asdas

7

Conclusiones y trabajos futuros

En este capítulo final se realiza un balance crítico del proyecto *Reset*. Se analiza el nivel de cumplimiento de los objetivos establecidos al inicio de la memoria, se proponen futuras líneas de desarrollo para la expansión del prototipo y se exponen las conclusiones personales del autor acerca del proceso de aprendizaje y el desarrollo multidisciplinar llevado a cabo.

7.1. Grado de Cumplimiento de los Objetivos

Como se ha establecido en el capítulo 2, el propósito fundamental de este Trabajo de Fin de Grado era el diseño y desarrollo de una demo jugable (*Vertical Slice*) de *Reset* que unificase diferentes perspectivas y géneros mecánicos bajo una misma arquitectura lógica. Para evaluar el éxito del proyecto, se desglosa a continuación el nivel de consecución de cada uno de los objetivos específicos planteados:

- **Elaboración de un Documento de Diseño de Juego (GDD) exhaustivo:** *Objetivo Cumplido*. Se ha realizado un documento completo de diseño y desarrollo del videojuego que explica claramente todo lo que ha abarcado el proyecto, desde las inspiraciones conceptuales y el diseño de la experiencia hasta el desarrollo técnico de los diferentes sistemas.
- **Desarrollo de una narrativa cohesiva como hilo conductor:** *Objetivo Cumplido*. A nivel del prototipo se ha establecido una narrativa sólida con

visión a futuro, logrando una buena conexión con los usuarios que han participado en las pruebas de validación. La integración del Hub o Nexo central funciona como el núcleo narrativo que justifica la transición del jugador entre mundos. A través del gestor de diálogos y la interacción con terminales y personajes emblemáticos, se ha logrado que el cambio de perspectiva de cámara y de mecánicas de juego se perciba como un viaje coherente y no como una colección de minijuegos inconexos.

- **Implementación de un diseño de niveles estructurado en capas:** *Objetivo casi cumplido.* El juego introduce una curva de aprendizaje progresiva. El Mundo 1 plantea un entorno de acción y exploración accesible centrado en el combate cuerpo a cuerpo y a distancia, mientras que el Mundo 2 eleva el reto mecánico en su segundo nivel. Aunque este diseño ofrece un desafío de precisión que satisface el interés de los jugadores experimentados, las pruebas demostraron que la curva de dificultad resultó demasiado de imprevisto y pronunciada para los usuarios noveles en sus fases iniciales.
- **Programación de un motor de físicas y control en 2D adaptado al género de plataformas:** *Objetivo casi cumplido.* Se ha diseñado y programado un controlador físico completo aplicando sistemas específicos como el búfer de entrada (*jump buffer*), el tiempo de gracia (*coyote time*) y el salto variable. La única limitación encontrada ha sido cierta inestabilidad física al interactuar con los suelos traspasables nativos del motor Unity, los cuales generaron colisiones imprecisas en situaciones de alta velocidad aérea.
- **Desarrollo de una arquitectura escalable de Inteligencia Artificial:** *Objetivo Cumplido.* Los infectados del Mundo 1 se estructuraron bajo una jerarquía orientada a objetos sólida. El comportamiento de los enemigos de combate y la evitación dinámica de obstáculos mediante navegación local por direccionamiento de contexto (*Context Steering*) demostraron que es posible lograr comportamientos orgánicos y eficientes sin depender de mallas de navegación complejas. Aunque el sistema es susceptible de recibir refinamientos algorítmicos en el futuro, el grado de cumplimiento de esta meta es muy alto.
- **Implementación de un sistema robusto de persistencia de datos y gestión de estado:** *Objetivo Cumplido.* Se diseñó una arquitectura de persistencia modular en tres bloques lógicos (**PlayerPrefs** para opciones globales de audio, serialización JSON para el progreso del personaje, e identificadores únicos basados en coordenadas físicas para el escenario). Este sistema ha demostrado ser seguro y coherente ante la transición entre escenas y las subrutinas de muerte y reinicio limpio de la partida.
- **Integración de sistemas mecánicos específicos de cada género:** *Objetivo Cumplido.* Se implementaron lógicas a medida como el Director de

Drops probabilístico adaptativo (DDA) en el Mundo 1, los cristales de recarga y el movimiento inercial sobre plataformas móviles en el Mundo 2, dotando a cada entorno de la robustez mecánica propia de sus géneros de referencia.

7.2. Trabajo Futuro y Líneas de Ampliación

Aunque la versión actual de *Reset* cumple con los requisitos lúdicos y de ingeniería de una demo funcional, el proyecto se ha estructurado de forma modular para permitir una expansión escalable en el futuro, posibilitando completar la ambiciosa visión inicial de la obra. Las principales líneas de ampliación propuestas son:

1. **Ampliación de Contenido y Mundos:** Diseñar e implementar nuevos mundos de juego con mecánicas y perspectivas alternativas, tales como una perspectiva en primera persona para exploración tridimensional o una vista isométrica táctica. Esto enriquecerá el concepto original de hibridación de géneros, creando una amplia biblioteca de mundos que eleven la jugabilidad a un nivel donde la narrativa adquiera una escala global.
2. **Pulido Visual y Animación:** Enriquecer el apartado artístico *pixel art* mediante la adición de más estéticas diferentes con mejores acabados, más detalles que respondan principalmente a la experiencia de usuario (UX) y el pulido de animaciones, *sprites* y, en líneas generales, de todo el apartado visual para profesionalizarlo.
3. **Continuación Narrativa y Psicología del Jugador:** Seguir redactando la historia en base al prototipo para completar el hilo narrativo del Nexo. Se plantea realizar un estudio más profundo de la psicología del jugador durante la navegación para redactar una continuación que mantenga al usuario conectado, persiguiendo un nivel superior de narrativa que diferencie claramente a la obra en el mercado.
4. **Refinamiento del Sistema de Colisiones:** Sustituir los colisionadores de plataforma nativos de Unity por un algoritmo de detección por trazado de rayos personalizado (*Raycasting*) en el controlador del personaje, solucionando de forma definitiva los problemas de colisiones que podían provocar errores o atascos en los diferentes niveles.
5. **Comercialización y Marketing:** En definitiva, completar el desarrollo de la demo jugable hasta consolidar una experiencia de juego pulida y atractiva con el fin de promocionarla, publicar el proyecto en plataformas de distribución digital (como itch.io o Steam) y profesionalizar el trabajo realizado.

7.3. Conclusiones Personales

Bibliografía

- [1] Unity Technologies, “Unity game engine,” 2024, accedido el 6 de julio de 2026. [Online]. Available: <https://unity.com/>
- [2] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [3] A. Fray, “Context steering: Behavior-driven ai,” 2013, publicado en Game AI Pro. [Online]. Available: <http://www.gameai.pro.com/>
- [4] Extremely OK Games, “Celeste (videojuego),” 2018.
- [5] Naughty Dog, “The last of us (videojuego),” 2013.
- [6] Capcom, “Resident evil (videojuego),” 1996.
- [7] Nintendo, “Super mario bros. (videojuego),” 1985.

Apéndice



Este es el primer apéndice

A.1. Ejemplo de funcionamiento de LaTeX

Esto es una referencia bibliográfica [?]. Se recomienda leer “The Not So Short Introduction to L^AT_EX” [?] (existen versiones más modernas).

A.1.1. Tablas y figuras

Las tablas y figuras deben presentarse en el texto, referenciadas y numeradas. La descripción de una figura debe ir posicionada debajo de la misma. Las descripciones de tablas pueden aparecer encima o debajo de las mismas (pero de forma consistente en todo el documento).

Tabla A.1: Título de la tabla.

	Subs.	Students	A	PE	WA	RE	CTE	IF	TLE	All
Ex. 1	104	44	1.27	0	0.55	0.23	0.20	0.11	0	2.36
Ex. 2	118	37	0.92	0	0.92	0.27	0.49	0.59	0	3.19
Ex. 3	100	28	1.21	0.39	1.18	0.54	0.14	0.07	0.04	3.57
Ex. 4	78	25	1.08	0.84	0.52	0.40	0.24	0.04	0	3.12
Ex. 5	116	31	1.48	0.10	0.77	0.32	0.42	0.19	0.45	3.74
Ex. 6	213	32	1.06	0.34	3.81	0.56	0.69	0.06	0.13	6.66
Ex. 7	116	34	1.35	0.38	0.38	0.68	0.62	0	0	3.41
Average	120.7	33	1.20	0.26	1.14	0.42	0.40	0.16	0.08	3.66

Tabla A.2: Tabla rotada. Factor groupings for the Mooshak questionnaire.

Factor	Interpretation / Items* (loadings)	Median	Mode
1	Students' perception of Mooshak towards its helpfulness in learning		
(21.17%) $\alpha = 0.922$	m10. Mooshak has forced me to implement programs more carefully (0,849)	4	4
	m6. Mooshak has helped me improve as a programmer (0,819)	3	4
	m5. Mooshak has made me more aware of the need to write correct code (0,781)	3	3
	m1. Mooshak has forced me to program more responsibly (0,713)	3	3
	m15. The specifications regarding the exercises used with Mooshak are adequate (0,687)	3	3
	m18. Mooshak helps to measure my current programming skills (0,680)	2.5	3
2	Disposition towards using Mooshak		
(17.93%) $\alpha = 0.897$	m24. I would be willing to participate in a programming contest using Mooshak, with similar exercises to the ones seen throughout the course (0,807)	2	1
	m13. Using Mooshak in the final exams is a good idea (0,748)	2	1
	m14. I would like to use Mooshak or a similar tool in the future (0,734)	3	1
	m17. Knowing Mooshak can motivate me to take part in a programming contest (0,655)	2	1
	m9. It would have been useful to use Mooshak from the first programming course (0,527)	2.5	1
	m16. Using Mooshak in the course has been interesting (0,522)	3	4
3	Effect of Mooshak's feedback in the tool's usefulness		
(14.84%) $\alpha = 0.836$	m12. Mooshak's feedback is adequate (0,832)	2	1
	m3. Using Mooshak has increased my workload considerably (0,693)	4	4
	m7. If Mooshak does not accept my code I feel motivated to find and fix the errors (0,691)	2	3
	m8. In general, using Mooshak has been a good idea (0,666)	3	4
4	Mooshak's effect on persistence		
(11.20%) $\alpha = 0.705$	m23. When Mooshak does not accept my code I get discouraged and I abandon the exercise (0,848)	3	3
	m22. Mooshak has been a waste of time (0,597)	2	2
	m25. Once a program has passed Mooshak's tests, I rewrite it in order to enhance it (0,559)	2	2
5	Students' perception of Mooshak's features		
(10.87%) $\alpha = 0.742$	m20. Even if it is not related to the grade, I feel satisfied if I am one of the first students to complete an exercise (0,729)	2	2
	m19. I value the fact that a tool like Mooshak returns feedback in real time about the correction of my programs (0,650)	3.5	4

*Measured on a 5-point Likert scale (1: strongly disagree; 2: disagree; 3: neutral; 4: agree; 5: strongly agree).

numeric literals	integers	in decimal	8743
		in octal	0o7464
			00103
		in hexadecimal	0x5A0FF
	0xE0F2		
	fractionals	in decimal	140.58
			8.04e7
			0.347E+12
5.47E-12			
47e22			
char literals			'H'
			'\n'
			'\x65'
string literals			"bom dia"
			"ouro preto\nmg"

Tabla A.3: Tabla con “multicolumnas” y “multifilas”.



Figura A.1: Logo de la Universidad.

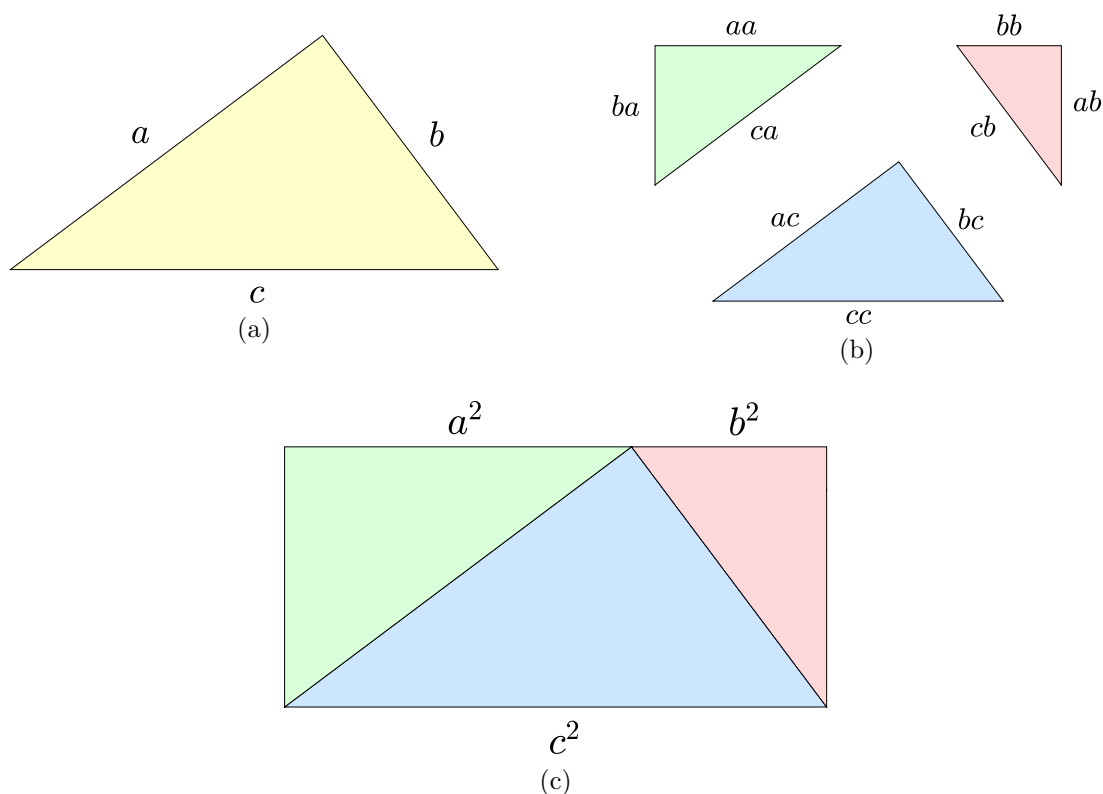


Figura A.2: Ejemplo con varias figuras. Demostración visual del teorema de Pitágoras. En (a) tenemos un triángulo rectángulo con hipotenusa c y catetos a y b . En (b) se muestra tres copias escaladas del mismo triángulo. El verde se ha escalado por a , el rojo/rosa por b , y el azul por c . En (c) se juntan los triángulos de (b) para formar un rectángulo cuya base es c^2 , pero también $a^2 + b^2$. Por tanto, $a^2 + b^2 = c^2$.

En las tablas se recomienda evitar líneas verticales y usar pocas horizontales.

La figura A.1 se utiliza en la portada. L^AT_EX ubica automáticamente las tablas y figuras. Para ello emplea reglas basadas en la experiencia de profesionales de la edición de textos. Podemos forzar su ubicación, pero en general es recomendable usar la ubicación sugerida por el sistema L^AT_EX. Usad gráficos vectoriales siempre que podáis.

A.2. Segunda sección

Normalmente no tendremos que insertar saltos de página, salvo para forzar que los capítulos empiecen en páginas impares, con

`\blankpage`

En cualquier caso, podemos introducir un salto de página con el comando

`\newpage`

.

Código A.1: Título del algoritmo/código.

```

1 def sum_list_limits_1(a, lower, upper):
2     if lower > upper:
3         return 0
4     else:
5         return a[upper] + sum_list_limits_1(a, lower, upper -
        1)

```

Algoritmo 1 *Additional Louvain* **input**=(A , $\mathcal{M}$) **output**= P

```

1:  $\forall i \in V$ , let  $i$  be an isolated community
2:  $o = \text{permutation}(V)$ 
3: for  $k \in o$  do
4:   search in  $A$  all the neighbours of  $k$ ,  $j$ 
5:    $\forall j$ , calculate  $\Delta Q_k(j)$  in matrix  $\mathcal{M}$ 
6:    $j^* = \{ j \mid \Delta Q_k(j^*) = \max_j \{Q_k(j)\} \}$ 
7:   if  $\Delta Q_k(j^*) > 0$  then
8:     Move node  $k$  to  $j^*$  's community
9:   else
10:     $k$  remains in its community
11:   end if
12: end for

```

A.2.1. Código

El código A.1 es un ejemplo en Python.

Código A.2: Endpoint REST para recuperar un libro por su id

```

1 @GetMapping("/{id}")
2 public ResponseEntity<Book> getBook(@PathVariable long id) {
3     // Comment to explain the code
4     Optional<Book> op = service.findById(id);
5     log.info("Book with id {} found", id);
6     if (op.isPresent()) {
7         Book book = op.get();
8         return new ResponseEntity<>(book, HttpStatus.OK);
9     } else {
10        return new ResponseEntity<>(HttpStatus.NOT_FOUND);
11    }
12 }

```

El código A.2 es un ejemplo en Java.

En el algoritmo 1 aparece un ejemplo en pseudocódigo.

A.2.2. Comentarios

Esto es un ejemplo de texto Tutor ▷ *Esto es un comentario del tutor* ◁ Alumno
▷ *Esto es un comentario del alumno* ◁